Manual paso a paso

Creación de la app móvil **Pizza Julián** en Flutter con base de datos PostgreSQL, servidor API en Dart y Docker Desktop



Proyecto: **D:\FLUTTER\flutter**
Repositorio: github.com/delfinalber/FLUTTER
Flutter 3.47.9 · Dart 3.13.7 · PostgreSQL 16 · Docker Desktop

Colombia · 8 de noviembre de 2026

Contenido

Introducción: qué se construyó y cómo está organizado.

I. **Parte 1. Preparar el computador e instalar Flutter** (pasos 1 a 6)
II. **Parte 2. Docker Desktop y el contenedor de PostgreSQL** (pasos 7 a 9)
III. **Parte 3. Crear la base de datos en PostgreSQL** (pasos 10 a 18)
IV. **Parte 4. Crear y ejecutar el servidor API en Dart** (pasos 19 a 24)
V. **Parte 5. Crear la app en Flutter** (pasos 25 a 32)
VI. **Parte 6. Ejecutar la app y recorrer todas sus pantallas** (pasos 33 a 42)
VII. **Parte 7. Celular, GitHub y uso diario** (pasos 43 a 46)

Introducción

Este manual explica, paso a paso y con pantallazos de cada etapa, cómo se construyó la app móvil de **Pizza Julián** a partir del sitio web `pizza-julian` (hecho en PHP, Bootstrap y MariaDB). La app conserva el mismo diseño (banner, menú, carrusel, video, acordeón, productos, mapa, formulario de contacto, WhatsApp) y las mismas acciones (inicio de sesión con contador de 30 segundos y editores para crear, editar y eliminar contenido).

La solución tiene tres partes que trabajan juntas:



> **¿Por qué hay un servidor API?** Una app móvil no debe conectarse directamente a PostgreSQL, porque tendría que llevar la contraseña de la base de datos dentro del instalador (APK), y cualquiera podría sacarla. El servidor API es el único que habla con la base de datos; la app solo habla con el servidor, igual que antes el navegador solo hablaba con los archivos `.php`.

Estructura final de la carpeta del proyecto:

```
D:\FLUTTER\flutter
├── flutter_sdk\                      SDK de Flutter 3.47.9 (no se sube a GitHub)
├── base_de_datos\
│   └── pizza_julian_postgres.sql     Script de la base de datos PostgreSQL
├── crear_base_de_datos.ps1           Crea la base en el PostgreSQL de Windows
├── pizza_julian_api\                 Servidor API en Dart
│   ├── bin\server.dart               Punto de arranque
│   ├── lib\api.dart                  Rutas: tema, login, inicio, productos, contacto
│   ├── lib\paleta.dart               Paleta de colores calculada desde el banner
│   ├── lib\imagenes.dart             Validación y guardado de imágenes
│   ├── media\                        Imágenes del banner, carrusel y productos
│   └── .env                          Contraseñas (no se sube a GitHub)
├── pizza_julian_app\                 App Flutter
│   └── lib\  main.dart, api.dart, tema.dart, navegacion.dart, config.dart
│       ├── pantallas\  inicio, productos, contacto y sus tres editores
│       └── widgets\    banner, menú, pie de página, WhatsApp, carrusel, login, editor
├── README.md
└── Manual_App_Flutter_Pizza_Julian.pdf   Este manual
```

Parte 1. Preparar el computador e instalar Flutter

1 Revisar los requisitos

Antes de empezar, verifique que el computador tenga lo siguiente:

Programa	Para qué se usa	Dónde se consigue
Windows 10 u 11 (64 bits)	Sistema operativo del computador	—
Google Chrome	Ver y probar la app en el navegador	google.com/chrome
Visual Studio Code	Editar el código	code.visualstudio.com
Git	Guardar versiones y subir a GitHub	git-scm.com
Docker Desktop	Correr PostgreSQL en un contenedor (Parte 2)	docker.com/products/docker-desktop
PostgreSQL 16 (opcional)	Base de datos instalada directamente en Windows	postgresql.org/download/windows
Flutter SDK 3.47.9	Crear y compilar la app (incluye Dart)	Se instala en los pasos 2 y 3

2 Descargar y descomprimir el SDK de Flutter

Abra **PowerShell**, ubíquese en `D:\FLUTTER\flutter` y descargue la versión estable de Flutter para Windows. Luego compruebe que el archivo llegó completo (el código SHA-256 debe coincidir con el publicado por Flutter), descomprímalo y cambie el nombre de la carpeta a `flutter_sdk`:

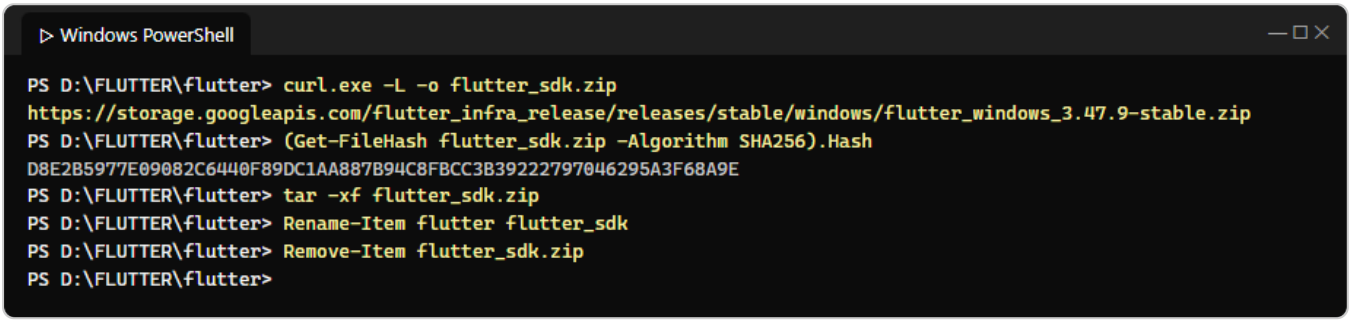


Figura 1. Descarga, verificación y descompresión del SDK de Flutter.

Al final, la carpeta `D:\FLUTTER\flutter\flutter_sdk` debe verse así:

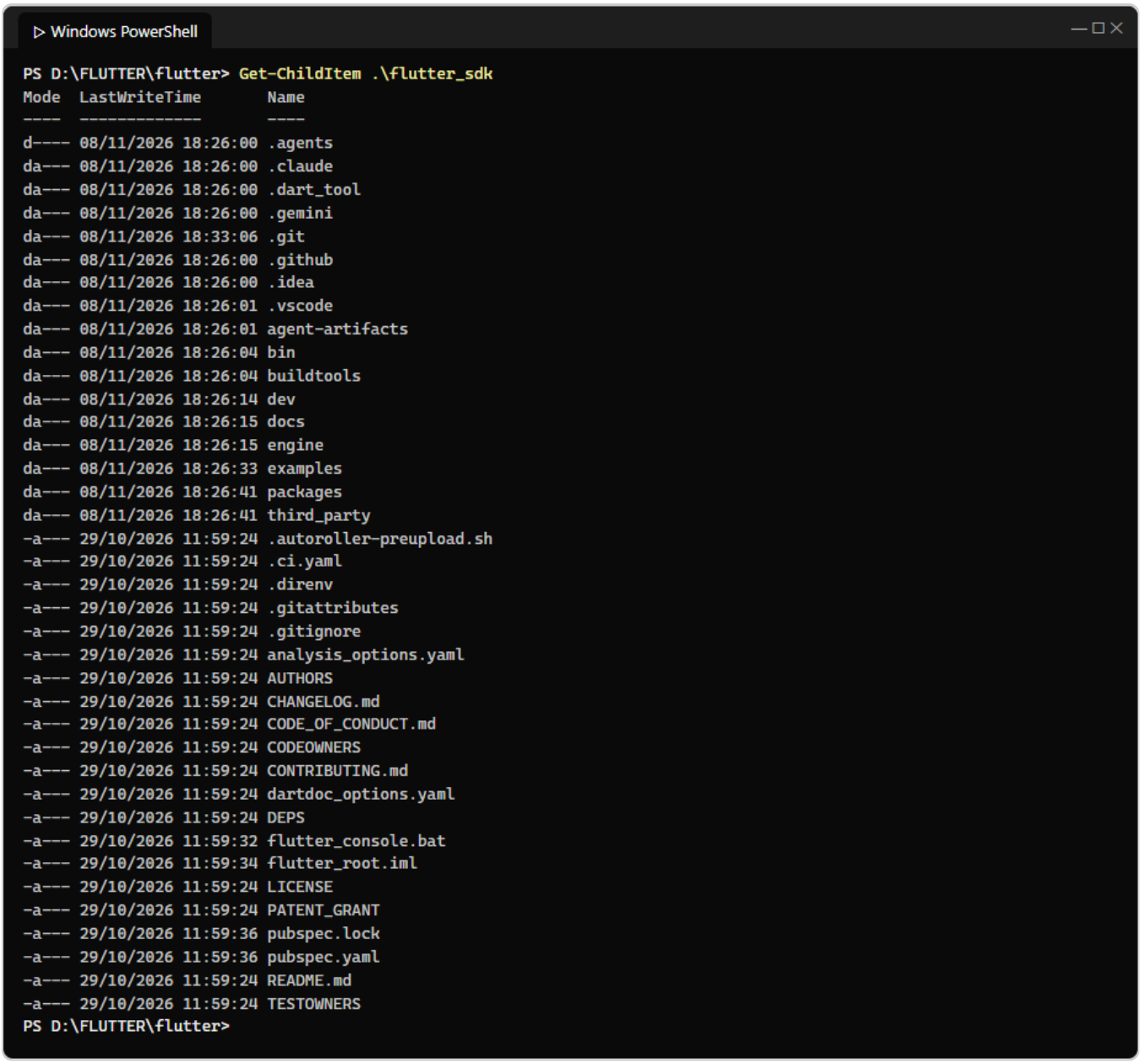


Figura 2. Contenido de la carpeta del SDK de Flutter.

3 Agregar Flutter al PATH de Windows

Para que los comandos `flutter` y `dart` funcionen desde cualquier carpeta, se agrega `D:\FLUTTER\flutter\flutter_sdk\bin` a la variable **Path** del usuario. Luego, con `where.exe`, se confirma que Windows encuentra los programas:



Figura 3. Flutter agregado al PATH y verificado con where.exe.

> Después de cambiar el PATH, **cierre y vuelva a abrir** PowerShell o Visual Studio Code para que tomen el cambio. También puede hacerlo por la interfaz: *Inicio → "Editar las variables de entorno de esta cuenta" → Path → Nuevo*.

4 Verificar la versión instalada

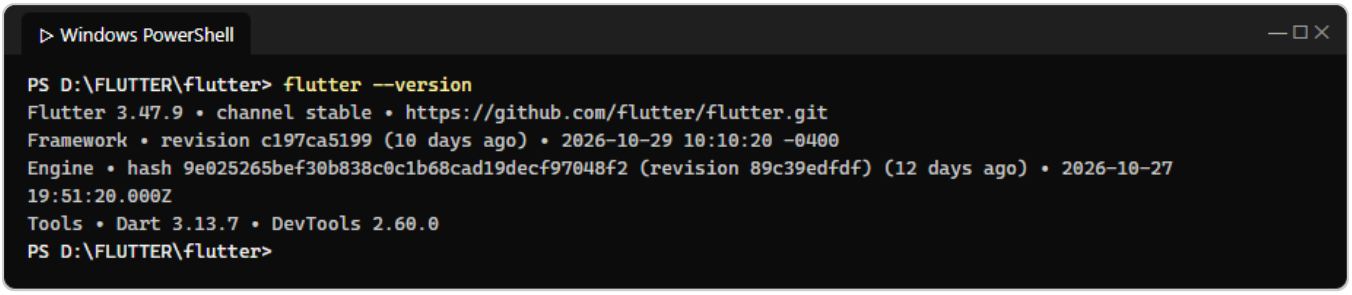


Figura 4. flutter --version muestra Flutter 3.47.9 y Dart 3.13.7.

5 Revisar el estado con flutter doctor

`flutter doctor` revisa todo lo que Flutter necesita. Lo importante para este proyecto es que aparezca con chulo **[√] Chrome**, para ver la app en el navegador.

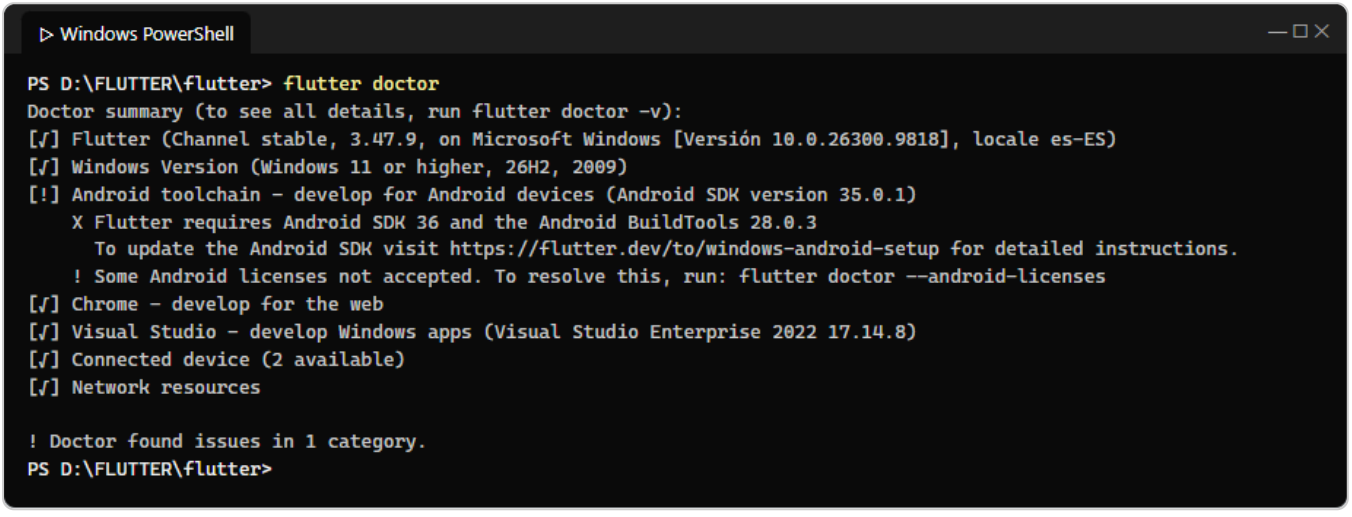


Figura 5. Resultado de flutter doctor.

El aviso **[!] Android toolchain** solo afecta a la compilación del instalador para celular (APK). Se soluciona abriendo **Android Studio → More Actions → SDK Manager**, instalando **Android SDK Platform 36** y **Build-Tools**, y luego ejecutando `flutter doctor --android-licenses` (responda `y` a todo).

6 Ver los dispositivos disponibles

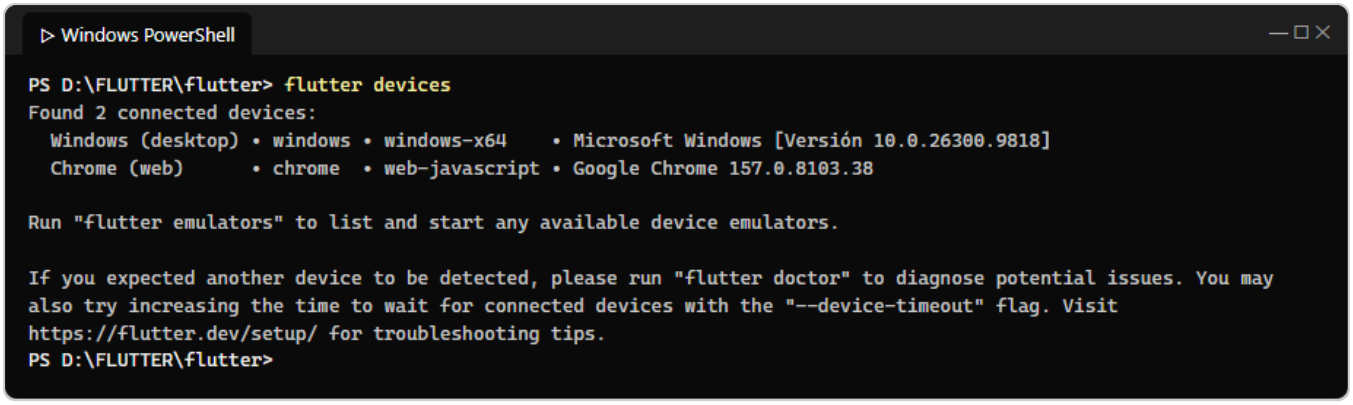


Figura 6. Flutter detecta Chrome (web) y Windows (escritorio).

Parte 2. Docker Desktop y el contenedor de PostgreSQL

Docker Desktop permite correr PostgreSQL dentro de un **contenedor**: un "mini computador" aislado que trae PostgreSQL ya instalado. Así no hace falta conocer ni cambiar la contraseña del PostgreSQL instalado en Windows, y se puede borrar y volver a crear en segundos.

7 Instalar, abrir y verificar Docker Desktop

1. Descargue Docker Desktop desde **docker.com/products/docker-desktop** e instálelo con las opciones por defecto (usa WSL 2).
2. Reinicie el computador si el instalador lo pide.
3. Abra **Docker Desktop** desde el menú Inicio y espere a que el ícono de la ballena, en la barra de tareas, quede quieto (dice *Engine running*).
4. Si prefiere la terminal, también puede encenderlo con `docker desktop start`.

Verifique que Docker responde:



Figura 7. Versión de Docker y estado "running" de Docker Desktop.

8 Descargar la imagen de PostgreSQL 16

Una **imagen** es la plantilla desde la que se crean los contenedores. Se usa `postgres:16-alpine`, que es PostgreSQL 16 en una versión liviana:

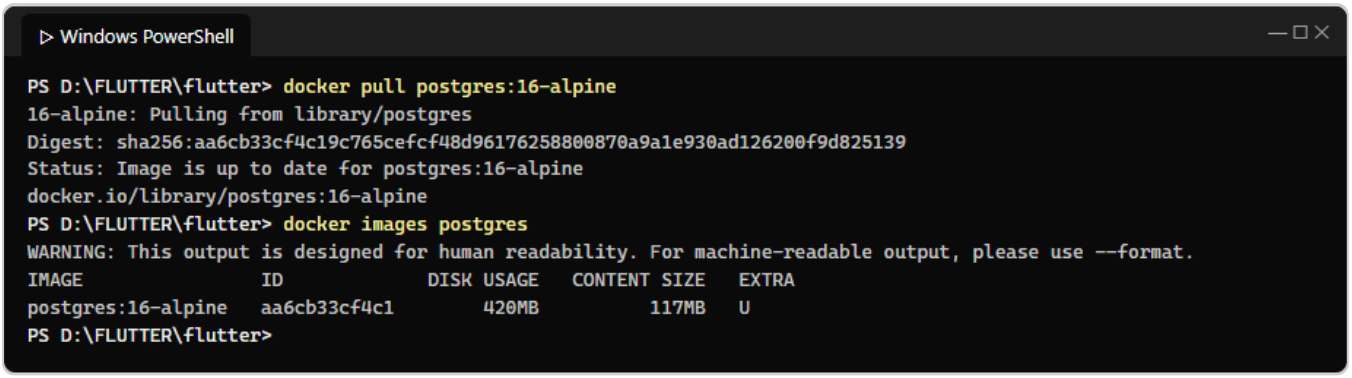


Figura 8. Descarga de la imagen postgres:16-alpine.

9 Crear el contenedor pizza_julian_db

Este comando crea y enciende el contenedor con la base de datos de la app:



Figura 9. Creación del contenedor, verificación con docker ps y volumen de datos.

Parte del comando	Qué significa
`--name pizza_julian_db`	Nombre del contenedor (así aparece en Docker Desktop → **Containers**).
`--restart unless-stopped`	Se enciende solo cada vez que abre Docker Desktop.
`-p 5433:5432`	El PostgreSQL del contenedor queda en el puerto **5433** del computador (el 5432 lo usa el PostgreSQL de Windows).
`-e POSTGRES_PASSWORD=pizzajulian`	Contraseña del usuario `postgres` dentro del contenedor.
`-e POSTGRES_DB=pizza_julian`	Crea la base de datos vacía `pizza_julian` al iniciar.
`-v pizza_julian_datos:/var/lib/...`	**Volumen**: los datos se guardan aunque se apague o se reinicie el contenedor.
`-v D:/FLUTTER/flutter/base_de_datos:/sql:ro`	Comparte la carpeta del script SQL con el contenedor (solo lectura).

En la ventana de Docker Desktop verá el contenedor en **Containers** → **pizza_julian_db** con el puerto **5433:5432**. Desde allí puede detenerlo (■), iniciarlo (▶), ver sus registros (*Logs*) o abrir una terminal (*Exec*). En **Volumes** aparece `pizza_julian_datos` y en **Images**, `postgres:16-alpine`.

Parte 3. Crear la base de datos en PostgreSQL

10 Analizar la base de datos original (MariaDB)

El punto de partida fue el archivo `pizza-julian/bd/tecnico-2026-pagina.sql`, exportado desde phpMyAdmin (MariaDB). Por ejemplo, así estaba la tabla de mensajes de contacto:

pizza-julian/bd/tecnico-2026-pagina.sql (MariaDB original) — Visual Studio Code

```
27  -- Estructura de tabla para la tabla `formulario-contacto`
28  --
29
30  CREATE TABLE `formulario-contacto` (
31    `id_formulario` int(11) NOT NULL,
32    `correo_formulario` varchar(150) NOT NULL,
33    `nombre_formulario` varchar(150) NOT NULL,
34    `telefono_formulario` bigint(13) NOT NULL,
35    `mensaje_formulario` text NOT NULL,
36    `fecha-formulario` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()
```

Figura 10. Tabla formulario-contacto en la base original de MariaDB.

Para pasarla a PostgreSQL se hicieron estos cambios:

MariaDB (original)	PostgreSQL (nuevo)	Motivo
`` `formulario-contacto` ``, `` `carru-1` ``	`formulario_contacto`, `carru_1`	PostgreSQL no acepta guiones en los nombres sin comillas.
`int(11) AUTO_INCREMENT`	`INTEGER GENERATED ALWAYS AS IDENTITY`	Forma estándar de los consecutivos en PostgreSQL.
`timestamp`	`TIMESTAMPTZ`	Fecha con zona horaria.
Contraseña `$2y$` (PHP)	`$2a$` + extensión `pgcrypto`	Mismo algoritmo bcrypt; la contraseña del usuario no cambia.
No existía	Tabla `contacto`	La usaba `contactos.php` para el mapa, pero no estaba en el archivo .sql.
Sesión PHP (cookie)	Tabla `sesion_api`	Guarda los "tokens" de inicio de sesión de la app (duran 8 horas).

11 Escribir el script pizza_julian_postgres.sql

El script está en `D:\FLUTTER\flutter\base_de_datos\pizza_julian_postgres.sql`. Empieza activando `pgcrypto`, borra las tablas si ya existían (para poder ejecutarlo varias veces) y crea la tabla de usuarios:

base_de_datos/pizza_julian_postgres.sql — Visual Studio Code

```sql
-- ================================================================
-- Base de datos PostgreSQL para la app móvil PIZZA JULIAN (Flutter)
-- Basada en: pizza-julian/bd/tecnico-2026-pagina.sql (MariaDB)
--
-- Cambios respecto a la versión MariaDB:
--   * Nombres con guion pasan a guion bajo (PostgreSQL no los admite sin comillas):
--       `formulario-contacto` -> formulario_contacto, `carru-1` -> carru_1, `fecha-formulario` ->
fecha_formulario
--   * AUTO_INCREMENT -> GENERATED ALWAYS AS IDENTITY
--   * Se agrega la tabla `contacto` (la usa contactos.php pero no estaba en el volcado)
--   * Se agrega la tabla `sesion_api` (tokens de inicio de sesión de la app móvil)
--   * Las contraseñas se verifican en la base con pgcrypto (bcrypt), igual que password_verify de PHP
--   * Las rutas de imágenes son relativas a la carpeta /media del servidor API
--
-- Uso (desde PowerShell):
--   & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -c "CREATE DATABASE pizza_julian ENCODING
'UTF8';"
--   & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d pizza_julian -f
base_de_datos\pizza_julian_postgres.sql
-- ================================================================

SET client_encoding = 'UTF8';

BEGIN;

CREATE EXTENSION IF NOT EXISTS pgcrypto;

DROP TABLE IF EXISTS sesion_api;
DROP TABLE IF EXISTS formulario_contacto;
DROP TABLE IF EXISTS contacto;
DROP TABLE IF EXISTS productos;
DROP TABLE IF EXISTS inicio;
DROP TABLE IF EXISTS usuario;

-- ----------------------------------------------------------------
-- Usuarios que pueden entrar a los editores de contenido
-- ----------------------------------------------------------------
CREATE TABLE usuario (
  id_usuario  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario     VARCHAR(200) NOT NULL UNIQUE,
  password    VARCHAR(200) NOT NULL -- hash bcrypt
);

-- Mismo usuario y hash del sitio web. PHP genera "$2y$" y pgcrypto usa "$2a$":
-- son el mismo algoritmo, por eso solo se cambia el prefijo y la contraseña sigue siendo la misma.
INSERT INTO usuario (usuario, password) VALUES
('alberdelfin', '$2a$10$eGR.EOb270JBg7t30CyVNO7Z9HK4j3L.APJsD12lsRX8OfGx4FzUG');

-- Para crear otro usuario o cambiar una contraseña:
--   INSERT INTO usuario (usuario, password) VALUES ('nuevo', crypt('su-clave', gen_salt('bf', 10)));
--   UPDATE usuario SET password = crypt('nueva-clave', gen_salt('bf', 10)) WHERE usuario = 'alberdelfin';
```

Figura 11. Inicio del script: extensión pgcrypto y tabla usuario.

base_de_datos/pizza_julian_postgres.sql — Visual Studio Code

```sql
-- ------------------------------------------------------------------------
-- Tokens de sesión de la app (reemplazan la sesión PHP con cookie)
-- ------------------------------------------------------------------------
CREATE TABLE sesion_api (
  token       CHAR(64) PRIMARY KEY,
  id_usuario  INTEGER NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
  creado      TIMESTAMPTZ NOT NULL DEFAULT now(),
  expira      TIMESTAMPTZ NOT NULL
);
CREATE INDEX idx_sesion_api_expira ON sesion_api (expira);

-- ------------------------------------------------------------------------
-- Contenido de la pantalla Inicio (el primer registro define banner y paleta de toda la app)
-- ------------------------------------------------------------------------
CREATE TABLE inicio (
  id_inicio               INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  banner_inicio           VARCHAR(200) NOT NULL DEFAULT '',
  paleta_banner           VARCHAR(400) NOT NULL DEFAULT '',
  carru_1                 VARCHAR(200) NOT NULL DEFAULT '',
  carru_2                 VARCHAR(200) NOT NULL DEFAULT '',
  carru_3                 VARCHAR(200) NOT NULL DEFAULT '',
  url_video_inicio        VARCHAR(200) NOT NULL DEFAULT '',
  acor1_titulo_inicio     VARCHAR(150) NOT NULL DEFAULT '',
  acor1_texto_inicio      VARCHAR(500) NOT NULL DEFAULT '',
  acor2_titulo_inicio     VARCHAR(150) NOT NULL DEFAULT '',
  acor2_texto_inicio      VARCHAR(500) NOT NULL DEFAULT '',
  acor3_titulo_inicio     VARCHAR(150) NOT NULL DEFAULT '',
  acor3_texto_inicio      VARCHAR(500) NOT NULL DEFAULT '',
  acor4_titulo_inicio     VARCHAR(150) NOT NULL DEFAULT '',
  acor4_texto_inicio      VARCHAR(500) NOT NULL DEFAULT '',
  acor5_titulo_inicio     VARCHAR(150) NOT NULL DEFAULT '',
  acor5_texto_inicio      VARCHAR(500) NOT NULL DEFAULT '',
  colapsar1_titulo_inicio VARCHAR(150) NOT NULL DEFAULT '',
  colapsar1_texto_inicio  VARCHAR(500) NOT NULL DEFAULT '',
  colapsar2_titulo_inicio VARCHAR(150) NOT NULL DEFAULT '',
  colapsar2_texto_inicio  VARCHAR(500) NOT NULL DEFAULT '',
  colapsar3_titulo_inicio VARCHAR(150) NOT NULL DEFAULT '',
  colapsar3_texto_inicio  VARCHAR(500) NOT NULL DEFAULT '',
  numero_whatsapp         VARCHAR(20)  NOT NULL DEFAULT '',
  actualizado             TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

Figura 12. Tablas sesion_api (tokens de la app) e inicio (contenido de la pantalla principal).

base_de_datos/pizza_julian_postgres.sql — Visual Studio Code

```sql
-- ------------------------------------------------------------------------
-- Tarjetas de la pantalla Productos (cada una con un carrusel de 3 imágenes)
-- ------------------------------------------------------------------------
CREATE TABLE productos (
  id_producto            INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  img1_carru_productos   VARCHAR(200) NOT NULL,
  img2_carru_productos   VARCHAR(200) NOT NULL,
  img3_carru_productos   VARCHAR(200) NOT NULL,
  card_title_productos   VARCHAR(200) NOT NULL,
  card_texto_productos   VARCHAR(500) NOT NULL,
  btn_primary_productos  VARCHAR(200) NOT NULL
);

-- Productos que el sitio tenía publicados en productos.html
INSERT INTO productos (img1_carru_productos, img2_carru_productos, img3_carru_productos, card_title_productos,
card_texto_productos, btn_primary_productos) VALUES
('productos/pizza1.png',   'productos/pizza2.png',   'productos/pizza3.png',   'Pizza Napolitana',
'Salsa de tomate, anchoas, ajo, orégano y aceite de oliva.', 'Pedir ahora'),
('productos/pizza4.jpeg',  'productos/pizza5.png',   'productos/pizza6.png',   'Pizza Boloñesa',
'Salsa boloñesa de carne, mozzarella y orégano.', 'Pedir ahora'),
('productos/pizza7.jpeg',  'productos/pizza8.png',   'productos/pizza9.png',   'Pizza Hawaiana',
'Jamón, piña, mozzarella y salsa de tomate.', 'Pedir ahora'),
('productos/pizza10.jpeg', 'productos/pizza11.png',  'productos/pizza12.png',  'Pizza Vegetariana',
'Pimientos, cebolla, champiñones, aceitunas y tomates cherry.', 'Pedir ahora'),
```

Figura 13. Tabla productos y carga de las 16 pizzas que tenía la página web.

base_de_datos/pizza_julian_postgres.sql — Visual Studio Code

```sql
-- ------------------------------------------------------------------------
-- Mapa de la pantalla Contacto
-- ------------------------------------------------------------------------
CREATE TABLE contacto (
  id_contacto   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  url_contacto  VARCHAR(1000) NOT NULL
);

INSERT INTO contacto (url_contacto) VALUES
('https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d18953.883843598873!2d-
75.2785308969908!3d2.9410217172291766!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x8e3b7464ac59aa5b%3A0x7e513d685
43b6eec!2sDomino''s%20Pizza%20Las%20Brisas!5e0!3m2!1ses-419!2sco!4v1784748693148!5m2!1ses-419!2sco');

-- ------------------------------------------------------------------------
-- Mensajes que llegan desde el formulario de contacto
-- ------------------------------------------------------------------------
CREATE TABLE formulario_contacto (
  id_formulario        INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  correo_formulario    VARCHAR(150) NOT NULL,
  nombre_formulario    VARCHAR(150) NOT NULL,
  telefono_formulario  VARCHAR(20)  NOT NULL DEFAULT '',
  mensaje_formulario   TEXT         NOT NULL,
  fecha_formulario     TIMESTAMPTZ  NOT NULL DEFAULT now()
);

INSERT INTO formulario_contacto (correo_formulario, nombre_formulario, telefono_formulario, mensaje_formulario,
fecha_formulario) VALUES
('delfin.alber@gmail.com', 'Prueba Automatica', '3132345685', 'Mensaje de prueba desde agente', '2026-08-28
14:37:37+00'),
('delfin.alber@gmail.com', 'DANNA LISETH PULIDO ZABALETA', '3132345685', 'Hola', '2026-08-28 14:40:29+00'),
('delfin.alber@gmail.com', 'ALBER DELFIN PEÑA ORTIGOZA', '3132345685', 'Hola un saludo', '2026-08-28
14:48:52+00'),
('g46113026@gmail.com', 'CUALQUIERA', '3213386504', 'Hola checho estamos en el Tecnico Programando con PHP.',
'2026-09-02 18:30:40+00');

COMMIT;
```

Figura 14. Tablas contacto (mapa) y formulario_contacto (mensajes), y cierre con COMMIT.

12 Verificar el cliente psql

`psql` es la consola de PostgreSQL. Con Docker se usa la que trae el contenedor (`docker exec pizza_julian_db psql ...`):

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql --version
psql (PostgreSQL) 16.15
PS D:\FLUTTER\flutter>
```

Figura 15. Versión de psql: PostgreSQL 16.

13 Crear la base de datos pizza_julian

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -c "CREATE DATABASE pizza_julian ENCODING 'UTF8'
TEMPLATE template0;"
CREATE DATABASE
PS D:\FLUTTER\flutter>
```

Figura 16. CREATE DATABASE pizza_julian con codificación UTF-8 (para tildes y la ñ).

> **Si va a usar el PostgreSQL instalado en Windows** (en lugar de Docker), todo esto lo hace el archivo `crear_base_de_datos.ps1`. Ejecútelo en PowerShell desde `D:\FLUTTER\flutter` con `.\crear_base_de_datos.ps1`. Le pedirá la contraseña del usuario `postgres`, la misma que usa en pgAdmin.

crear_base_de_datos.ps1 — Visual Studio Code

```
1  # Crea la base de datos PostgreSQL "pizza_julian" con todas las tablas y datos iniciales.
2  # Uso (PowerShell, desde D:\FLUTTER\flutter):   .\crear_base_de_datos.ps1
3  # Pide la contraseña del usuario postgres que se definió al instalar PostgreSQL.
4
5  $psql = "C:\Program Files\PostgreSQL\16\bin\psql.exe"
6  if (-not (Test-Path $psql)) {
7      $psql = (Get-Command psql -ErrorAction SilentlyContinue).Source
8  }
9  if (-not $psql) {
10     Write-Host "No se encontró psql.exe. Instale PostgreSQL o ajuste la ruta en este script." -ForegroundColor
Red
11     exit 1
12 }
13
14 $script = Join-Path $PSScriptRoot "base_de_datos\pizza_julian_postgres.sql"
15
16 $existe = & $psql -U postgres -h localhost -tAc "SELECT 1 FROM pg_database WHERE datname = &#x27;pizza_julian'"
17 if ($existe -ne "1") {
18     & $psql -U postgres -h localhost -c "CREATE DATABASE pizza_julian ENCODING &#x27;UTF8' TEMPLATE template0;"
19 }
20 & $psql -U postgres -h localhost -d pizza_julian -v ON_ERROR_STOP=1 -f $script
21 if ($LASTEXITCODE -eq 0) {
22     Write-Host "Base de datos pizza_julian lista." -ForegroundColor Green
23 }
```

Figura 17. Script crear_base_de_datos.ps1 para el PostgreSQL de Windows.

14 Ejecutar el script para crear las tablas y los datos

Se ejecuta el archivo `.sql` dentro de la base. Los avisos *NOTICE: table … does not exist, skipping* son normales la primera vez: indican que no había tablas viejas que borrar. Al final debe aparecer **COMMIT**:

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -v ON_ERROR_STOP=1 -f
/sql/pizza_julian_postgres.sql
SET
BEGIN
CREATE EXTENSION
psql:/sql/pizza_julian_postgres.sql:25: NOTICE:  table "sesion_api" does not exist, skipping
DROP TABLE
DROP TABLE
psql:/sql/pizza_julian_postgres.sql:26: NOTICE:  table "formulario_contacto" does not exist, skipping
DROP TABLE
psql:/sql/pizza_julian_postgres.sql:27: NOTICE:  table "contacto" does not exist, skipping
psql:/sql/pizza_julian_postgres.sql:28: NOTICE:  table "productos" does not exist, skipping
DROP TABLE
psql:/sql/pizza_julian_postgres.sql:29: NOTICE:  table "inicio" does not exist, skipping
DROP TABLE
DROP TABLE
psql:/sql/pizza_julian_postgres.sql:30: NOTICE:  table "usuario" does not exist, skipping
CREATE TABLE
INSERT 0 1
CREATE TABLE
CREATE INDEX
CREATE TABLE
INSERT 0 1
CREATE TABLE
INSERT 0 16
CREATE TABLE
INSERT 0 1
CREATE TABLE
INSERT 0 4
COMMIT
PS D:\FLUTTER\flutter>
```

Figura 18. Ejecución del script: 6 tablas creadas, 16 productos y 4 mensajes cargados.

15 Verificar las tablas creadas

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "\dt"
              List of relations
 Schema |        Name         | Type  |  Owner
--------+---------------------+-------+----------
 public | contacto            | table | postgres
 public | formulario_contacto | table | postgres
 public | inicio              | table | postgres
 public | productos           | table | postgres
 public | sesion_api          | table | postgres
 public | usuario             | table | postgres
(6 rows)
PS D:\FLUTTER\flutter>
```

Figura 19. El comando \dt lista las 6 tablas de la base pizza_julian.

16 Revisar la estructura de cada tabla

Con `\d nombre_tabla` se ven las columnas, los tipos y las llaves:

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "\d productos"
                                    Table "public.productos"
        Column         |          Type          | Collation | Nullable |           Default
-----------------------+------------------------+-----------+----------+-----------------------------
 id_producto           | integer                |           | not null | generated always as identity
 img1_carru_productos  | character varying(200) |           | not null |
 img2_carru_productos  | character varying(200) |           | not null |
 img3_carru_productos  | character varying(200) |           | not null |
 card_title_productos  | character varying(200) |           | not null |
 card_texto_productos  | character varying(500) |           | not null |
 btn_primary_productos | character varying(200) |           | not null |
Indexes:
    "productos_pkey" PRIMARY KEY, btree (id_producto)
PS D:\FLUTTER\flutter>
```

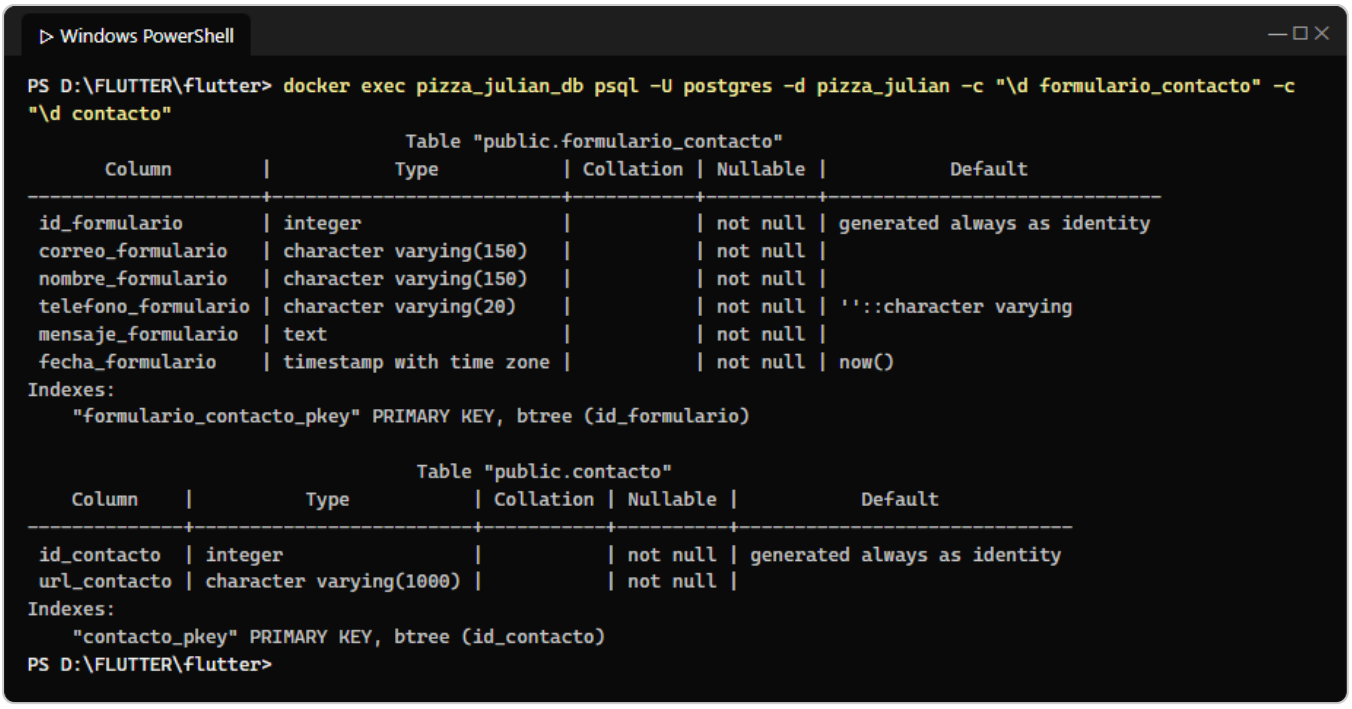
Figura 20. Estructura de la tabla productos.

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "\d usuario" -c "\d
sesion_api"
                                Table "public.usuario"
   Column   |          Type          | Collation | Nullable |           Default
------------+------------------------+-----------+----------+-----------------------------
 id_usuario | integer                |           | not null | generated always as identity
 usuario    | character varying(200) |           | not null |
 password   | character varying(200) |           | not null |
Indexes:
    "usuario_pkey" PRIMARY KEY, btree (id_usuario)
    "usuario_usuario_key" UNIQUE CONSTRAINT, btree (usuario)
Referenced by:
    TABLE "sesion_api" CONSTRAINT "sesion_api_id_usuario_fkey" FOREIGN KEY (id_usuario) REFERENCES
usuario(id_usuario) ON DELETE CASCADE

                    Table "public.sesion_api"
   Column   |           Type           | Collation | Nullable | Default
------------+--------------------------+-----------+----------+---------
 token      | character(64)            |           | not null |
 id_usuario | integer                  |           | not null |
 creado     | timestamp with time zone |           | not null | now()
 expira     | timestamp with time zone |           | not null |
Indexes:
    "sesion_api_pkey" PRIMARY KEY, btree (token)
    "idx_sesion_api_expira" btree (expira)
Foreign-key constraints:
    "sesion_api_id_usuario_fkey" FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
PS D:\FLUTTER\flutter>
```

Figura 21. Tablas usuario y sesion_api (con su llave foránea hacia usuario).

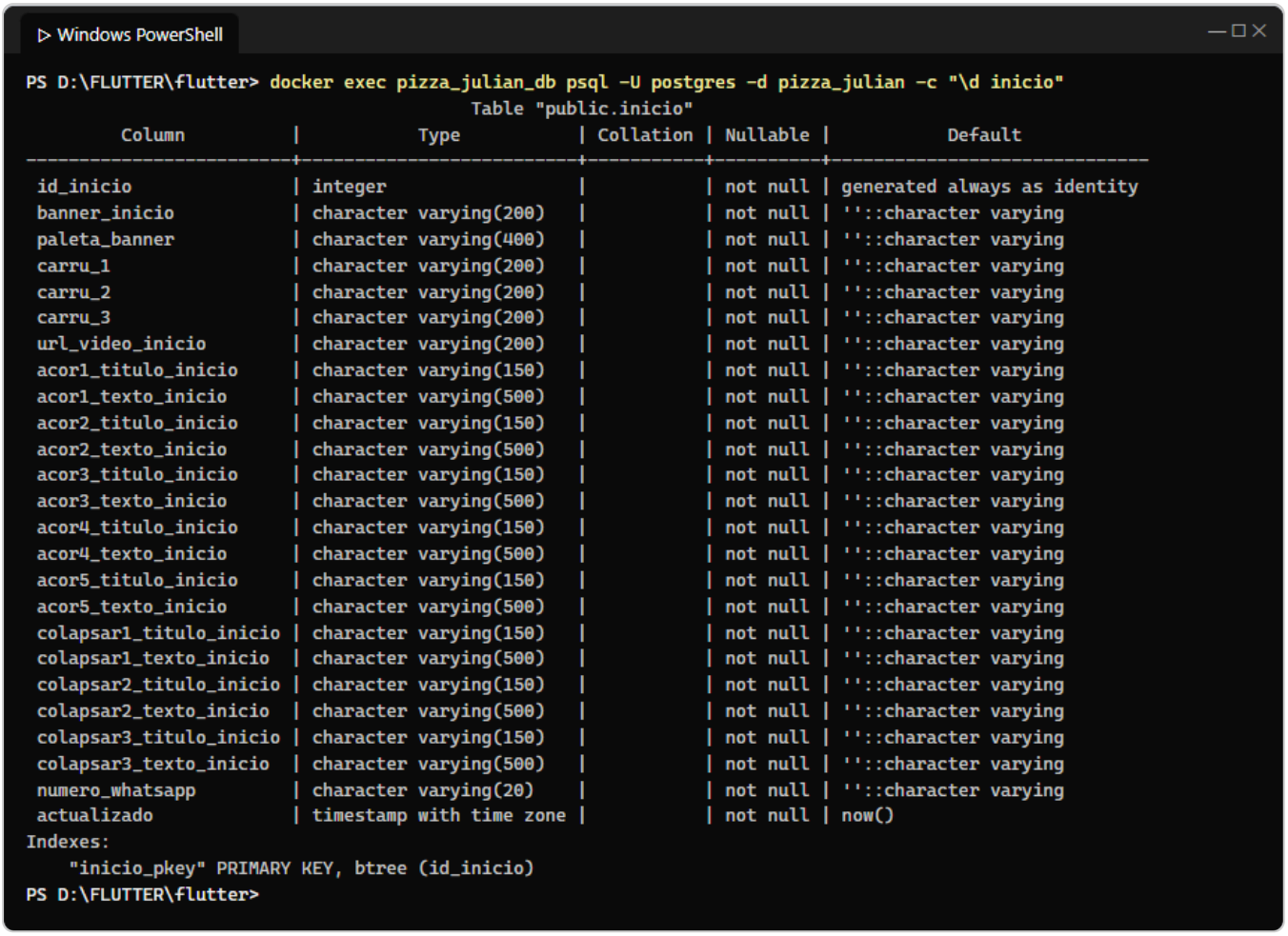


```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "\d formulario_contacto" -c
"\d contacto"
                                Table "public.formulario_contacto"
       Column        |           Type           | Collation | Nullable |           Default
---------------------+--------------------------+-----------+----------+------------------------------
 id_formulario       | integer                  |           | not null | generated always as identity
 correo_formulario   | character varying(150)   |           | not null |
 nombre_formulario   | character varying(150)   |           | not null |
 telefono_formulario | character varying(20)    |           | not null | ''::character varying
 mensaje_formulario  | text                     |           | not null |
 fecha_formulario    | timestamp with time zone |           | not null | now()
Indexes:
    "formulario_contacto_pkey" PRIMARY KEY, btree (id_formulario)

                                Table "public.contacto"
    Column    |          Type           | Collation | Nullable |           Default
--------------+-------------------------+-----------+----------+------------------------------
 id_contacto  | integer                 |           | not null | generated always as identity
 url_contacto | character varying(1000) |           | not null |
Indexes:
    "contacto_pkey" PRIMARY KEY, btree (id_contacto)
PS D:\FLUTTER\flutter>
```

Figura 22. Tablas formulario_contacto y contacto.



```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "\d inicio"
                                    Table "public.inicio"
         Column          |           Type           | Collation | Nullable |           Default
-------------------------+--------------------------+-----------+----------+------------------------------
 id_inicio               | integer                  |           | not null | generated always as identity
 banner_inicio           | character varying(200)   |           | not null | ''::character varying
 paleta_banner           | character varying(400)   |           | not null | ''::character varying
 carru_1                 | character varying(200)   |           | not null | ''::character varying
 carru_2                 | character varying(200)   |           | not null | ''::character varying
 carru_3                 | character varying(200)   |           | not null | ''::character varying
 url_video_inicio        | character varying(200)   |           | not null | ''::character varying
 acor1_titulo_inicio     | character varying(150)   |           | not null | ''::character varying
 acor1_texto_inicio      | character varying(500)   |           | not null | ''::character varying
 acor2_titulo_inicio     | character varying(150)   |           | not null | ''::character varying
 acor2_texto_inicio      | character varying(500)   |           | not null | ''::character varying
 acor3_titulo_inicio     | character varying(150)   |           | not null | ''::character varying
 acor3_texto_inicio      | character varying(500)   |           | not null | ''::character varying
 acor4_titulo_inicio     | character varying(150)   |           | not null | ''::character varying
 acor4_texto_inicio      | character varying(500)   |           | not null | ''::character varying
 acor5_titulo_inicio     | character varying(150)   |           | not null | ''::character varying
 acor5_texto_inicio      | character varying(500)   |           | not null | ''::character varying
 colapsar1_titulo_inicio | character varying(150)   |           | not null | ''::character varying
 colapsar1_texto_inicio  | character varying(500)   |           | not null | ''::character varying
 colapsar2_titulo_inicio | character varying(150)   |           | not null | ''::character varying
 colapsar2_texto_inicio  | character varying(500)   |           | not null | ''::character varying
 colapsar3_titulo_inicio | character varying(150)   |           | not null | ''::character varying
 colapsar3_texto_inicio  | character varying(500)   |           | not null | ''::character varying
 numero_whatsapp         | character varying(20)    |           | not null | ''::character varying
 actualizado             | timestamp with time zone |           | not null | now()
Indexes:
    "inicio_pkey" PRIMARY KEY, btree (id_inicio)
PS D:\FLUTTER\flutter>
```

Figura 23. Tabla inicio: banner, carrusel, video, acordeón, secciones y WhatsApp.

17 Consultar los datos cargados

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "SELECT id_producto,
card_title_productos, img1_carru_productos FROM productos ORDER BY id_producto;"
 id_producto |    card_title_productos    |  img1_carru_productos
-------------+----------------------------+------------------------
           1 | Pizza Napolitana           | productos/pizza1.png
           2 | Pizza Boloñesa             | productos/pizza4.jpeg
           3 | Pizza Hawaiana             | productos/pizza7.jpeg
           4 | Pizza Vegetariana          | productos/pizza10.jpeg
           5 | Pizza Ranchera             | productos/pizza13.jpg
           6 | Pizza Tocineta             | productos/pizza16.png
           7 | Pizza Aceitunas            | productos/pizza19.jpeg
           8 | Pizza de Todito            | productos/pizza22.jpeg
           9 | Pizza Trufa Andina         | productos/pizza25.jpeg
          10 | Pizza Mediterranea Fina    | productos/pizza28.jpeg
          11 | Pizza BBQ Ahumada          | productos/pizza31.jpeg
          12 | Pizza Serrana              | productos/pizza43.png
          13 | Pizza Cuatro Quesos Deluxe | productos/pizza34.jpeg
          14 | Pizza Carbonara            | productos/pizza37.png
          15 | Pizza Criolla Picante      | productos/pizza40.png
          16 | Pizza Marinera             | productos/pizza46.jpeg
(16 rows)
PS D:\FLUTTER\flutter>
```

Figura 24. Los 16 productos con su título y su primera imagen.

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "SELECT id_inicio,
banner_inicio, acor1_titulo_inicio, numero_whatsapp FROM inicio;"
 id_inicio |   banner_inicio   |  acor1_titulo_inicio   | numero_whatsapp
-----------+-------------------+------------------------+-----------------
         1 | inicio/banner.png | Margarita (Margherita) | 573132345685
(1 row)
PS D:\FLUTTER\flutter>
```

Figura 25. Registro de la pantalla Inicio.

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "SELECT id_formulario,
nombre_formulario, correo_formulario, fecha_formulario FROM formulario_contacto;"
 id_formulario |      nombre_formulario       |   correo_formulario    |    fecha_formulario
---------------+------------------------------+------------------------+------------------------
             1 | Prueba Automatica            | delfin.alber@gmail.com | 2026-08-28 14:37:37+00
             2 | DANNA LISETH PULIDO ZABALETA | delfin.alber@gmail.com | 2026-08-28 14:40:29+00
             3 | ALBER DELFIN PEÑA ORTIGOZA   | delfin.alber@gmail.com | 2026-08-28 14:48:52+00
             4 | CUALQUIERA                   | g46113026@gmail.com    | 2026-09-02 18:30:40+00
(4 rows)
PS D:\FLUTTER\flutter>
```

Figura 26. Mensajes de contacto traídos de la base original.



Figura 27. Usuario alberdelfin; la contraseña se guarda cifrada (bcrypt), nunca en texto plano.

18 Crear un usuario administrador para los editores

Para entrar a los editores de la app se necesita un usuario. La función `crypt(..., gen_salt('bf', 10))` de `pgcrypto` cifra la contraseña. La consulta siguiente comprueba que la clave quedó bien guardada (`t` = verdadero):

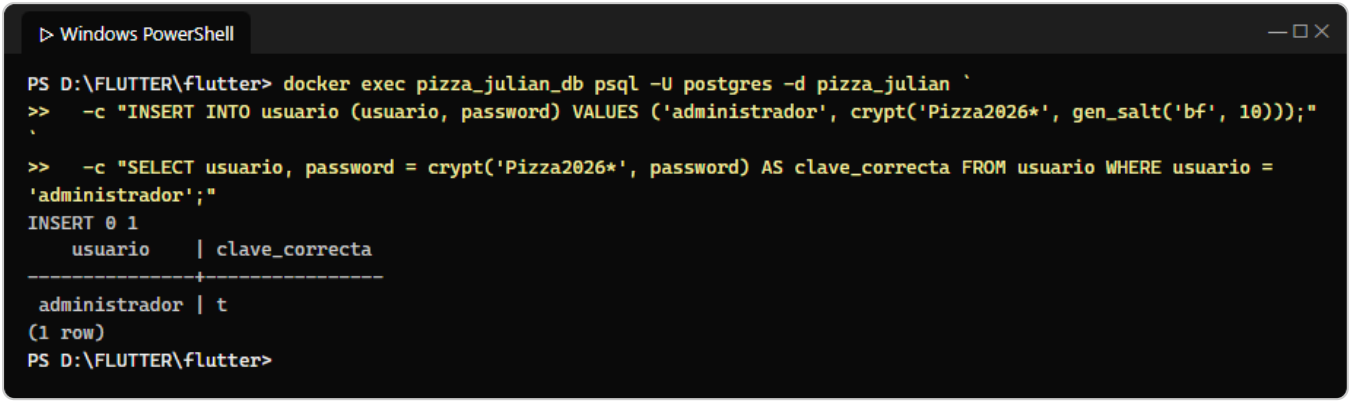


Figura 28. Creación del usuario administrador y verificación de su contraseña.

> El usuario `administrador` con la clave `Pizza2026*` se creó solo para las pruebas de este manual, en la base de Docker. En la base definitiva cree sus propios usuarios con una clave segura, y nunca la escriba en documentos ni en chats.

Parte 4. Crear y ejecutar el servidor API en Dart

19 Crear el proyecto del servidor y sus dependencias

El servidor está en `D:\FLUTTER\flutter\pizza_julian_api` y está escrito en Dart (el mismo lenguaje de Flutter). Su archivo `pubspec.yaml` declara los paquetes que usa:

pizza_julian_api/pubspec.yaml — Visual Studio Code

```yaml
name: pizza_julian_api
description: API REST en Dart para la app movil Pizza Julian (PostgreSQL).
version: 1.0.0
publish_to: none

environment:
  sdk: ">=3.4.0 <4.0.0"

dependencies:
  shelf: ^1.4.1
  shelf_router: ^1.1.4
  shelf_static: ^1.1.2
  postgres: ^3.4.0
  image: ^4.2.0
  mailer: ^6.1.2

dev_dependencies:
  lints: ^4.0.0
```

Figura 29. pubspec.yaml del servidor.

Paquete	Para qué sirve
`shelf`, `shelf_router`	Servidor web y rutas (`/api/productos`, `/api/login`...).
`shelf_static`	Entrega las imágenes de la carpeta `media`.
`postgres`	Conexión con PostgreSQL.
`image`	Valida las imágenes subidas y calcula la paleta de colores del banner.
`mailer`	Envía por Gmail los mensajes del formulario de contacto.

20 Configurar el archivo .env

Copie `.env.example` como `.env` y complete los datos. Con Docker use `DB_PORT=5433` y `DB_PASSWORD=pizzajulian`; con el PostgreSQL de Windows use `DB_PORT=5432` y la contraseña de `postgres`:

pizza_julian_api/.env.example — Visual Studio Code

```
# Copie este archivo como .env y complete los datos
# Conexión a PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_NAME=pizza_julian
DB_USER=postgres
DB_PASSWORD=

# Puerto del servidor API
PORT=8080

# Correo del formulario de contacto (contraseña de aplicación de Gmail)
GMAIL_USERNAME=delfin.alber@gmail.com
GMAIL_APP_PASSWORD=
CORREO_DESTINO=delfin.alber@gmail.com
```

Figura 30. Plantilla .env.example con las variables del servidor.

> El archivo `.env` contiene contraseñas, así que está en el `.gitignore` y **no se sube a GitHub**. Nunca lo comparta.

21 Programar el servidor

`bin/server.dart` lee la configuración, se conecta a PostgreSQL y abre el puerto 8080:

pizza_julian_api/bin/server.dart — Visual Studio Code

```dart
import 'dart:io';

import 'package:pizza_julian_api/api.dart';
import 'package:pizza_julian_api/config.dart';
import 'package:postgres/postgres.dart';
import 'package:shelf/shelf_io.dart' as io;

Future<void> main() async {
  final config = Config.cargar();

  final db = Pool.withEndpoints(
    [
      Endpoint(
        host: config.dbHost,
        port: config.dbPort,
        database: config.dbName,
        username: config.dbUser,
        password: config.dbPassword,
      ),
    ],
    settings: const PoolSettings(maxConnectionCount: 5, sslMode: SslMode.disable),
  );

  // Verifica la conexión antes de aceptar peticiones
  try {
    await db.execute('SELECT 1');
  } catch (e) {
    stderr.writeln('No fue posible conectar con PostgreSQL
(${config.dbHost}:${config.dbPort}/${config.dbName}): $e');
    stderr.writeln('Revise el archivo .env (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD).');
    exit(1);
  }

  final api = PizzaJulianApi(config, db);
  // 0.0.0.0 para que el celular o el emulador puedan llegar al servidor por la red local
  final servidor = await io.serve(api.handler, InternetAddress.anyIPv4, config.puerto);
  servidor.autoCompress = true;
  print('API Pizza Julian escuchando en http://${servidor.address.host}:${servidor.port}');
}
```

Figura 31. Punto de arranque del servidor (bin/server.dart).

En `lib/api.dart` están las rutas, que reemplazan a cada archivo PHP del sitio web:

pizza_julian_api/lib/api.dart (rutas) — Visual Studio Code

```dart
  Handler get handler {
    final router = Router()
      ..get('/api/salud', (Request r) => _json({'ok': true}))
      ..get('/api/tema', _tema)
      ..post('/api/login', _login)
      ..post('/api/logout', _logout)
      ..get('/api/sesion', _sesion)
      ..get('/api/inicio', _inicioPublico)
      ..get('/api/inicio/registros', _inicioRegistros)
      ..post('/api/inicio', (Request r) => _guardarInicio(r, null))
      ..put('/api/inicio/<id>', (Request r, String id) => _guardarInicio(r, int.tryParse(id) ?? 0))
      ..delete('/api/inicio/<id>', _eliminarInicio)
      ..get('/api/productos', _productos)
      ..post('/api/productos', (Request r) => _guardarProducto(r, null))
      ..put('/api/productos/<id>', (Request r, String id) => _guardarProducto(r, int.tryParse(id) ?? 0))
      ..delete('/api/productos/<id>', _eliminarProducto)
      ..get('/api/contacto', _contacto)
      ..post('/api/contacto', (Request r) => _guardarContacto(r, null))
      ..put('/api/contacto/<id>', (Request r, String id) => _guardarContacto(r, int.tryParse(id) ?? 0))
      ..post('/api/contacto/mensaje', _mensajeContacto)
      ..mount('/media/', createStaticHandler(config.carpetaMedia));

    return const
Pipeline().addMiddleware(logRequests()).addMiddleware(_cors()).addMiddleware(_errores()).addHandler(router.call);
  }
```

Figura 32. Rutas de la API.

Ruta	Reemplaza a	Qué hace
`GET /api/tema`	tema.php	Banner, paleta de colores y número de WhatsApp.
`GET /api/inicio`	index.php	Contenido de la pantalla Inicio.
`GET /api/productos`	productos.php	Lista de productos.
`GET /api/contacto`	contactos.php	URL del mapa.
`POST /api/contacto/mensaje`	formulario-contacto.php	Guarda el mensaje y lo envía por Gmail.
`POST /api/login` y `/api/logout`	login.php y logout.php	Inicio y cierre de sesión (bloqueo de 5 minutos tras 5 intentos fallidos).
`POST/PUT/DELETE /api/inicio`, `/api/productos`, `/api/contacto`	*-formulario.php	Crear, editar y eliminar (solo con sesión iniciada).

pizza_julian_api/lib/api.dart (inicio de sesión) — Visual Studio Code

```dart
  Future<Response> _login(Request request) async {
    final info = request.context['shelf.io.connection_info'] as HttpConnectionInfo?;
    final ip = info?.remoteAddress.address ?? 'desconocida';
    final intentos = _intentosPorIp.putIfAbsent(ip, _Intentos.new);

    // Bloqueo temporal tras varios intentos fallidos (mitiga fuerza bruta)
    if (DateTime.now().isBefore(intentos.bloqueoHasta)) {
      return _error('Demasiados intentos fallidos. Intente de nuevo en unos minutos.', estado: 429);
    }

    final datos = await _cuerpo(request);
    final usuario = _texto(datos, 'usuario');
    final password = (datos['password'] ?? '').toString();
    if (usuario.isEmpty || password.isEmpty || usuario.length > 200 || password.length > 200) {
      return _error('Usuario y contraseña son obligatorios.');
    }

    // crypt() de pgcrypto compara contra el hash bcrypt guardado, nunca en texto plano
    final filas = await _consulta(
      'SELECT id_usuario, usuario FROM usuario WHERE usuario = @usuario AND password =
crypt(@password, password) LIMIT 1',
      {'usuario': usuario, 'password': password},
    );
    if (filas.isEmpty) {
      intentos.fallidos++;
      if (intentos.fallidos >= 5) {
        intentos.bloqueoHasta = DateTime.now().add(const Duration(minutes: 5));
        intentos.fallidos = 0;
      }
      return _error('Usuario o contraseña incorrectos.', estado: 401);
    }
    _intentosPorIp.remove(ip);

    final token = List.generate(32, (_) => _aleatorio.nextInt(256).toRadixString(16).padLeft(2,
'0')).join();
    await db.execute('DELETE FROM sesion_api WHERE expira <= now()');
    await db.execute(
      Sql.named('INSERT INTO sesion_api (token, id_usuario, expira) VALUES (@token, @id, @expira)'),
      parameters: {'token': token, 'id': filas.first['id_usuario'],
'expira': DateTime.now().toUtc().add(_duracionSesion)},
    );
    return _json({'ok': true, 'mensaje': 'Inicio de sesión correcto.',
'token': token, 'usuario': filas.first['usuario']});
  }
```

Figura 33. Inicio de sesión: la contraseña se compara en PostgreSQL con crypt() y se genera un token.

22 Descargar los paquetes y revisar el código

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_api> dart pub get
Resolving dependencies...
Downloading packages...
  ffi 2.1.3 (2.2.0 available)
  intl 0.20.2 (0.20.3 available)
  lints 4.0.0 (6.1.0 available)
  mailer 6.6.0 (7.3.0 available)
  postgres 3.5.7 (3.6.0 available)
  shelf 1.4.2 (1.4.3 available)
  shelf_router 1.1.4 (1.1.5 available)
  shelf_static 1.1.3 (1.2.0 available)
Got dependencies!
8 packages have newer versions incompatible with dependency constraints.
Try `dart pub outdated` for more information.
PS D:\FLUTTER\flutter\pizza_julian_api>
```

Figura 34. dart pub get descarga los paquetes del servidor.

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_api> dart analyze
Analyzing pizza_julian_api...
No issues found!
PS D:\FLUTTER\flutter\pizza_julian_api>
```

Figura 35. dart analyze: "No issues found!" (sin errores).

23 Encender el servidor

```
cd D:\FLUTTER\flutter\pizza_julian_api
dart run bin/server.dart
```

Cuando aparece **"API Pizza Julian escuchando en http://0.0.0.0:8080"**, el servidor está listo. Cada línea siguiente es una petición que hizo la app (`[200]` significa que salió bien):

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_api> dart run bin/server.dart
API Pizza Julian escuchando en http://0.0.0.0:8080
2026-11-08T19:15:35.366158  0:00:00.011448 GET     [200] /api/salud
2026-11-08T19:16:08.002808  0:00:00.001182 OPTIONS [200] /api/tema
2026-11-08T19:16:08.009032  0:00:00.000399 OPTIONS [200] /api/inicio
2026-11-08T19:16:08.044848  0:00:00.029793 GET     [200] /media/inicio/banner.png
2026-11-08T19:16:08.013130  0:00:00.115973 GET     [200] /api/tema
2026-11-08T19:16:08.038013  0:00:00.301720 GET     [200] /api/inicio
2026-11-08T19:16:08.970517  0:00:00.001980 GET     [200] /media/inicio/1.jpeg
2026-11-08T19:16:13.961502  0:00:00.001410 GET     [200] /media/inicio/2.jpeg
2026-11-08T19:16:18.956746  0:00:00.001485 GET     [200] /media/inicio/3.jpeg
2026-11-08T19:16:37.543144  0:00:00.000311 OPTIONS [200] /api/tema
2026-11-08T19:16:37.545394  0:00:00.000328 OPTIONS [200] /api/inicio
PS D:\FLUTTER\flutter\pizza_julian_api>
```

Figura 36. Servidor encendido, recibiendo las peticiones de la app.

> Deje esta ventana de PowerShell **abierta** mientras usa la app. Para apagar el servidor presione **Ctrl + C**.

24 Probar la API en Chrome

Con el servidor encendido, abra estas direcciones en Chrome. Las respuestas son datos en formato JSON, sacados de PostgreSQL:

Dar formato al texto

{"ok":true}

Figura 37. http://localhost:8080/api/salud — el servidor responde.

Dar formato al texto

{"ok":true,"banner":"inicio/banner.png","paleta":{"negro":"#0f090a","negro-suave":"#1a1113","naranja":"#ff8b1a","amarillo":"#ffc21a","rojo":"#d90a0a","rojo-profundo":"#8f0909","rojo-vivo":"#ff2e2e","blanco":"#f7f5f3","gris":"#e7e2de"},"version":"30600886","whatsapp":"573132345685"}

Figura 38. http://localhost:8080/api/tema — banner, paleta de colores y WhatsApp.

Dar formato al texto


```
{"ok":true,"productos":
[{"id_producto":1,"img1_carru_productos":"productos/pizza1.png","img2_carru_productos":"productos/pizza2.png","img3_carru_productos":"produ
ctos/pizza3.png","card_title_productos":"Pizza Napolitana","card_texto_productos":"Salsa de tomate, anchoas, ajo, orégano y aceite de
oliva.","btn_primary_productos":"Pedir ahora"},
{"id_producto":2,"img1_carru_productos":"productos/pizza4.jpeg","img2_carru_productos":"productos/pizza5.png","img3_carru_productos":"produ
ctos/pizza6.png","card_title_productos":"Pizza Boloñesa","card_texto_productos":"Salsa boloñesa de carne, mozzarella y
orégano.","btn_primary_productos":"Pedir ahora"},
{"id_producto":3,"img1_carru_productos":"productos/pizza7.jpeg","img2_carru_productos":"productos/pizza8.png","img3_carru_productos":"produ
ctos/pizza9.png","card_title_productos":"Pizza Hawaiana","card_texto_productos":"Jamón, piña, mozzarella y salsa de
tomate.","btn_primary_productos":"Pedir ahora"},
{"id_producto":4,"img1_carru_productos":"productos/pizza10.jpeg","img2_carru_productos":"productos/pizza11.png","img3_carru_productos":"pro
ductos/pizza12.png","card_title_productos":"Pizza Vegetariana","card_texto_productos":"Pimientos, cebolla, champiñones, aceitunas y tomates
cherry.","btn_primary_productos":"Pedir ahora"},
{"id_producto":5,"img1_carru_productos":"productos/pizza13.jpg","img2_carru_productos":"productos/pizza14.png","img3_carru_productos":"prod
uctos/pizza15.png","card_title_productos":"Pizza Ranchera","card_texto_productos":"Carne, maíz tierno, tocineta y salsa
ranchera.","btn_primary_productos":"Pedir ahora"},
{"id_producto":6,"img1_carru_productos":"productos/pizza16.png","img2_carru_productos":"productos/pizza17.png","img3_carru_productos":"prod
uctos/pizza18.png","card_title_productos":"Pizza Tocineta","card_texto_productos":"Tocineta crocante, mozzarella y salsa de la
casa.","btn_primary_productos":"Pedir ahora"},
{"id_producto":7,"img1_carru_productos":"productos/pizza19.jpeg","img2_carru_productos":"productos/pizza20.png","img3_carru_productos":"pro
ductos/pizza21.png","card_title_productos":"Pizza Aceitunas","card_texto_productos":"Aceitunas negras y verdes, mozzarella y
orégano.","btn_primary_productos":"Pedir ahora"},
{"id_producto":8,"img1_carru_productos":"productos/pizza22.jpeg","img2_carru_productos":"productos/pizza23.png","img3_carru_productos":"pro
ductos/pizza24.png","card_title_productos":"Pizza de Todito","card_texto_productos":"Un poco de todo: carnes, vegetales y mucho
queso.","btn_primary_productos":"Pedir ahora"},
{"id_producto":9,"img1_carru_productos":"productos/pizza25.jpeg","img2_carru_productos":"productos/pizza26.png","img3_carru_productos":"pro
ductos/pizza27.png","card_title_productos":"Pizza Trufa Andina","card_texto_productos":"Salsa blanca, queso mozzarella, champinones y toque
de trufa.","btn_primary_productos":"Pedir ahora"},
{"id_producto":10,"img1_carru_productos":"productos/pizza28.jpeg","img2_carru_productos":"productos/pizza29.png","img3_carru_productos":"pr
oductos/pizza30.jpeg","card_title_productos":"Pizza Mediterranea Fina","card_texto_productos":"Salsa pomodoro, aceitunas negras, tomate
seco y hojas de albahaca.","btn_primary_productos":"Pedir ahora"},
{"id_producto":11,"img1_carru_productos":"productos/pizza31.jpeg","img2_carru_productos":"productos/pizza32.png","img3_carru_productos":"pr
oductos/pizza33.jpeg","card_title_productos":"Pizza BBQ Ahumada","card_texto_productos":"Salsa BBQ casera, pollo ahumado, cebolla morada y
queso cheddar.","btn_primary_productos":"Pedir ahora"},
{"id_producto":12,"img1_carru_productos":"productos/pizza43.png","img2_carru_productos":"productos/pizza44.png","img3_carru_productos":"pro
```


Figura 39. http://localhost:8080/api/productos — productos guardados en PostgreSQL.

Dar formato al texto


```
{"ok":true,"id_contacto":1,"url_contacto":"https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d18953.883843598873!2d-
75.2785308969908!3d2.9410217172291766!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x8e3b7464ac59aa5b%3A0x7e513d68543b6eec!2sDomino's%
20Pizza%20Las%20Brisas!5e0!3m2!1ses-419!2sco!4v1784748693148!5m2!1ses-419!2sco"}
```


Figura 40. http://localhost:8080/api/contacto — URL del mapa de Google.



Figura 41. http://localhost:8080/media/productos/pizza2.png — las imágenes también las entrega el servidor.

Parte 5. Crear la app en Flutter

25 Crear el proyecto Flutter

Desde `D:\FLUTTER\flutter` se crea el proyecto para Android, iOS y web:



Figura 42. flutter create genera el proyecto pizza_julian_app (82 archivos).

26 Agregar las dependencias

pizza_julian_app/pubspec.yaml — Visual Studio Code

```yaml
name: pizza_julian_app
description: "App movil Pizza Julian"
publish_to: &#x27;none'
version: 1.0.0+1

environment:
  sdk: ">=3.5.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  url_launcher: ^6.3.0
  webview_flutter: ^4.9.0
  image_picker: ^1.1.2
  font_awesome_flutter: ^11.0.0
  web: ^1.1.0
  pointer_interceptor: ^0.10.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^4.0.0

flutter:
  uses-material-design: true
```

Figura 43. pubspec.yaml de la app.

Paquete	Para qué sirve en la app
`http`	Comunicarse con el servidor API.
`url_launcher`	Abrir WhatsApp, YouTube y Google Maps.
`webview_flutter`	Mostrar el mapa de Google dentro de la app en el celular.
`image_picker`	Elegir imágenes de la galería en los editores.
`font_awesome_flutter`	Íconos de Facebook, Instagram, X, WhatsApp y YouTube.
`web`, `pointer_interceptor`	Mostrar el mapa en la versión de navegador y que no tape los clics de las ventanas.

27 Organizar el código y configurar la dirección del servidor

```
pizza_julian_app\lib
├── main.dart            Arranque de la app y tema de colores
├── config.dart          Dirección del servidor API
├── api.dart             Cliente de la API y sesión del editor
├── tema.dart            Paleta de colores (como las variables --pizza-* del CSS)
├── navegacion.dart      Cambio entre pantallas
├── pantallas\
│   ├── inicio.dart, productos.dart, contacto.dart
│   └── editor_inicio.dart, editor_productos.dart, editor_contacto.dart
└── widgets\
    ├── comunes.dart     Banner, menú, pie de página, WhatsApp, carrusel, alertas
    ├── login_dialogo.dart  Ventana de inicio de sesión con contador de 30 s
    ├── editor.dart      Piezas de los editores (formularios, selector de imágenes)
    └── iframe*.dart     Mapa de Google en la versión web
```

pizza_julian_app/lib/config.dart — Visual Studio Code

```dart
// Dirección del servidor API (carpeta pizza_julian_api).
// - Emulador Android: http://10.0.2.2:8080 (valor por defecto)
// - Celular real: la IP del computador en la red Wi-Fi, por ejemplo http://192.168.1.10:8080
// Se cambia sin tocar el código con:  flutter run --dart-define=API_URL=http://192.168.1.10:8080
const String apiUrl = String.fromEnvironment('API_URL', defaultValue: 'http://10.0.2.2:8080');

// Las imágenes se guardan en el servidor como rutas relativas ("productos/pizza1.png")
String urlMedia(String ruta) => '$apiUrl/media/$ruta';
```

Figura 44. config.dart: dirección del servidor, configurable con --dart-define=API_URL.

28 Programar el arranque y la paleta de colores

pizza_julian_app/lib/main.dart — Visual Studio Code

```dart
import 'package:flutter/material.dart';

import 'navegacion.dart';
import 'tema.dart';

void main() {
  runApp(const PizzaJulianApp());
}

class PizzaJulianApp extends StatefulWidget {
  const PizzaJulianApp({super.key});

  @override
  State<PizzaJulianApp> createState() => _PizzaJulianAppState();
}

class _PizzaJulianAppState extends State<PizzaJulianApp> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    temaSitio.sincronizar();
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }

  // Al volver a la app se revisa si otro editor cambió el banner y la paleta
  @override
  void didChangeAppLifecycleState(AppLifecycleState estado) {
    if (estado == AppLifecycleState.resumed) {
      temaSitio.sincronizar();
    }
  }

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: temaSitio,
      builder: (context, _) => MaterialApp(
        title: 'PIZZA-JULIAN',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: pz.naranja, primary: pz.rojo, secondary: pz.naranja),
          scaffoldBackgroundColor: Colors.white,
          useMaterial3: true,
        ),
        home: paginaDe(Seccion.inicio),
      ),
    );
  }
}
```

Figura 45. main.dart: al volver a la app se revisa si el banner o los colores cambiaron.

pizza_julian_app/lib/tema.dart (paleta de colores) — Visual Studio Code

```dart
import 'package:flutter/material.dart';

import 'api.dart';

// Paleta de colores compartida por todas las pantallas (equivale a las variables --pizza-* de tema.css).
// Sale del banner que se sube en el editor de Inicio.
class Paleta {
  const Paleta({
    required this.negro,
    required this.negroSuave,
    required this.naranja,
    required this.amarillo,
    required this.rojo,
    required this.rojoProfundo,
    required this.rojoVivo,
    required this.blanco,
    required this.gris,
  });

  final Color negro;
  final Color negroSuave;
  final Color naranja;
  final Color amarillo;
  final Color rojo;
  final Color rojoProfundo;
  final Color rojoVivo;
  final Color blanco;
  final Color gris;

  static const porDefecto = Paleta(
    negro: Color(0xFF0F090A),
    negroSuave: Color(0xFF1A1113),
    naranja: Color(0xFFFF8B1A),
    amarillo: Color(0xFFFFC21A),
    rojo: Color(0xFFD90A0A),
    rojoProfundo: Color(0xFF8F0909),
    rojoVivo: Color(0xFFFF2E2E),
    blanco: Color(0xFFF7F5F3),
    gris: Color(0xFFE7E2DE),
  );

  static Color? _color(Object? hex) {
    if (hex is! String || !RegExp(r'^#[0-9a-fA-F]{6}$').hasMatch(hex)) {
      return null;
    }
```

Figura 46. tema.dart: los mismos colores del sitio web (negro, naranja, amarillo, rojo…).

29 Programar la conexión con el servidor

pizza_julian_app/lib/api.dart (cliente de la API) — Visual Studio Code

```dart
class Api {
  static const _tiempoEspera = Duration(seconds: 30);

  Map<String, String> get _cabeceras => {
        'content-type': 'application/json; charset=utf-8',
        if (sesion.token != null) 'authorization': 'Bearer ${sesion.token}',
      };

  Future<Map<String, dynamic>> _enviar(String metodo, String ruta, [Map<String, dynamic>? cuerpo]) async {
    final uri = Uri.parse('$apiUrl/api/$ruta');
    final http.Response respuesta;
    try {
      final peticion = http.Request(metodo, uri)..headers.addAll(_cabeceras);
      if (cuerpo != null) {
        peticion.body = jsonEncode(cuerpo);
      }
      respuesta = await http.Response.fromStream(await peticion.send().timeout(_tiempoEspera));
    } catch (_) {
      throw ApiError('Error de comunicación con el servidor.');
    }

    Map<String, dynamic> datos;
    try {
      datos = Map<String, dynamic>.from(jsonDecode(utf8.decode(respuesta.bodyBytes)) as Map);
    } catch (_) {
      throw ApiError('Respuesta no válida del servidor.', respuesta.statusCode);
    }
    if (respuesta.statusCode >= 400 || datos['ok'] != true) {
      if (respuesta.statusCode == 401 && sesion.activa) {
        sesion.cerrar();
      }
      throw ApiError((datos['mensaje'] ?? 'No fue posible completar la
operación.').toString(), respuesta.statusCode);
    }
    return datos;
  }

  List<Map<String, dynamic>> _lista(Object? valor) => (valor as List? ?? []).map((e) => Map<String,
dynamic>.from(e as Map)).toList();
```

Figura 47. api.dart: envía las peticiones al servidor con el token de sesión.

30 Programar las pantallas

Cada pantalla de la web tiene su equivalente en Flutter. Por ejemplo, la pantalla Inicio arma el carrusel, el video, el acordeón y las secciones desplegables con los datos del servidor:

pizza_julian_app/lib/pantallas/inicio.dart — Visual Studio Code

```dart
class PantallaInicio extends StatefulWidget {
  const PantallaInicio({super.key});

  @override
  State<PantallaInicio> createState() => _PantallaInicioState();
}

class _PantallaInicioState extends State<PantallaInicio> {
  Map<String, String>? _datos;
  String? _error;

  @override
  void initState() {
    super.initState();
    _cargar();
  }

  Future<void> _cargar() async {
    try {
      final datos = await api.inicio();
      if (mounted) setState(() => (_datos = datos, _error = null));
    } on ApiError catch (e) {
      if (mounted) setState(() => _error = e.mensaje);
    }
  }

  @override
  Widget build(BuildContext context) {
    final d = _datos;
    return PaginaPizzeria(
      seccion: Seccion.inicio,
      alRefrescar: _cargar,
      hijos: d == null
          ? [EstadoCarga(error: _error, alReintentar: _cargar)]
          : [
              Carrusel(imagenes: [d['carru_1']!, d['carru_2']!,
d['carru_3']!]),
              VideoYoutube(url: d['url_video_inicio']!),
              const SizedBox(height: 16),
              Padding(
                padding: const EdgeInsets.symmetric(horizontal: 12),
                child: AcordeonPizzeria(
                  elementos: [for (var i = 1; i <= 5; i++) (d['acor${i}_titulo_inicio']!,
d['acor${i}_texto_inicio']!)],
                ),
              ),
              const SizedBox(height: 16),
              Padding(
                padding: const EdgeInsets.symmetric(horizontal: 12),
                child: ColapsablesPizzeria(
                  elementos: [for (var i = 1; i <= 3; i++) (d['colapsar${i}_titulo_inicio']!,
d['colapsar${i}_texto_inicio']!)],
                ),
              ),
            ],
    );
  }
}

String? idYoutube(String url) =>
    RegExp(r'(?:youtu\.be/|youtube\.com/(?:embed/|watch\?v=|shorts/))([A-Za-z0-9_-]{6,})',
caseSensitive: false).firstMatch(url)?.group(1);

// Miniatura del video; al tocarla se abre en la app de YouTube
class VideoYoutube extends StatelessWidget {
  const VideoYoutube({super.key, required this.url});
```

Figura 48. pantallas/inicio.dart.

31 Dar los permisos de Android e iOS

La app necesita permiso de Internet, permiso para conectarse al servidor local por `http` y permiso para abrir enlaces de WhatsApp y Maps. En iOS también se pide acceso a la galería de fotos:

android/app/src/main/AndroidManifest.xml — Visual Studio Code

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Conexión con el servidor API -->
    <uses-permission android:name="android.permission.INTERNET"/>
    <application
        android:label="Pizza Julian"
        android:usesCleartextTraffic="true"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTop"
```

Figura 49. AndroidManifest.xml con el permiso de Internet.

32 Descargar paquetes, revisar el código y correr las pruebas

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_app> flutter pub get
Resolving dependencies...
Downloading packages...
  async 2.11.0 (2.13.1 available)
  boolean_selector 2.1.1 (2.1.2 available)
  cross_file 0.3.4+2 (0.4.0 available)
  file_selector_linux 0.9.3+2 (0.9.4+1 available)
  file_selector_macos 0.9.4+2 (0.9.5+2 available)
  file_selector_platform_interface 2.6.2 (2.7.0 available)
  file_selector_windows 0.9.3+4 (0.9.3+6 available)
  flutter_lints 4.0.0 (6.0.0 available)
  flutter_plugin_android_lifecycle 2.0.26 (2.0.35 available)
  http_parser 4.0.2 (4.1.2 available)
  image_picker 1.1.2 (1.2.4 available)
  image_picker_android 0.8.12+21 (0.8.13+25 available)
  image_picker_for_web 3.0.6 (3.1.1 available)
  image_picker_ios 0.8.12+2 (0.8.13+15 available)
  image_picker_linux 0.2.1+2 (0.2.2 available)
  image_picker_macos 0.2.1+2 (0.2.2+1 available)
  image_picker_platform_interface 2.10.1 (2.11.2 available)
  image_picker_windows 0.2.1+1 (0.2.2 available)
  lints 4.0.0 (6.1.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  source_span 1.10.0 (1.10.2 available)
  string_scanner 1.2.0 (1.4.1 available)
  term_glyph 1.2.1 (1.2.2 available)
  test_api 0.7.12 (0.7.15 available)
  url_launcher 6.3.1 (6.3.3 available)
  url_launcher_android 6.3.14 (6.3.33 available)
  url_launcher_ios 6.3.3 (6.4.2 available)
  url_launcher_linux 3.2.1 (3.2.3 available)
  url_launcher_macos 3.2.2 (3.2.6 available)
  url_launcher_web 2.3.3 (2.4.3 available)
  url_launcher_windows 3.1.4 (3.1.6 available)
  vm_service 14.2.5 (15.4.0 available)
  web 1.1.1 (1.1.2 available)
  webview_flutter 4.10.0 (4.14.1 available)
  webview_flutter_android 4.3.2 (4.14.1 available)
  webview_flutter_platform_interface 2.13.0 (2.15.1 available)
  webview_flutter_wkwebview 3.22.0 (3.27.2 available)
Got dependencies!
37 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
PS D:\FLUTTER\flutter\pizza_julian_app>
```

Figura 50. flutter pub get.

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_app> flutter analyze
Analyzing pizza_julian_app...
No issues found! (ran in 18.0s)
PS D:\FLUTTER\flutter\pizza_julian_app>
```

Figura 51. flutter analyze: "No issues found!".

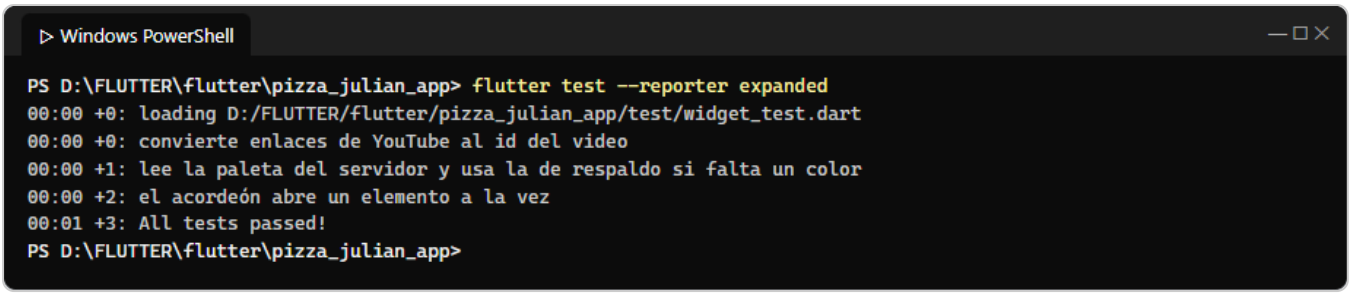


Figura 52. flutter test: las 3 pruebas automáticas pasan.

Parte 6. Ejecutar la app y recorrer todas sus pantallas

33 Abrir la app en Chrome

Con Docker Desktop abierto y el servidor encendido (paso 23), abra **otra** ventana de PowerShell y ejecute:

```
cd D:\FLUTTER\flutter\pizza_julian_app
flutter run -d chrome --dart-define=API_URL=http://localhost:8080
```

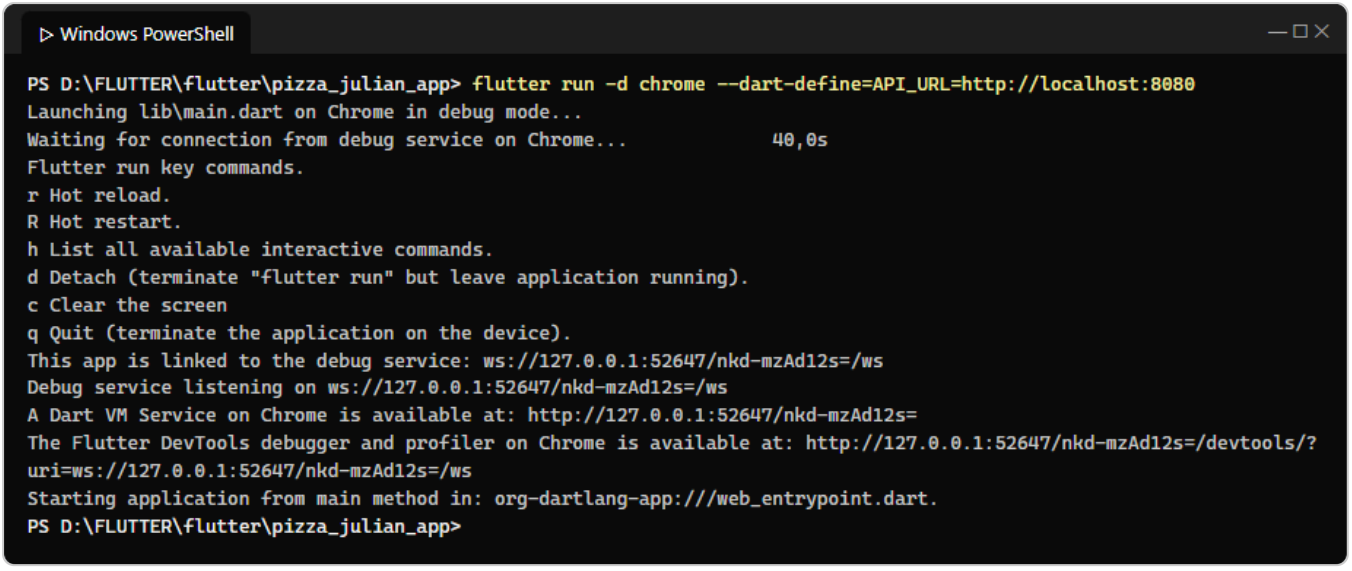


Figura 53. flutter run abre Chrome con la app y muestra las teclas disponibles.

> **Para verla con tamaño de celular:** en Chrome presione **F12** y luego **Ctrl + Shift + M**, y arriba elija un modelo (por ejemplo, "Pixel 7"). Mientras `flutter run` está abierto, si cambia el código presione **r** para recargar al instante y **q** para cerrar la app.

34 Pantalla Inicio y menú

Arriba se ve el banner y la barra con el menú. El botón ☰ despliega las opciones **Inicio**, **Productos**, **Contacto** y el botón **Inicio Sesion**. Más abajo están el carrusel (cambia solo cada 5 segundos) y el video de YouTube, que se abre en la app de YouTube. El botón redondo de WhatsApp abre el chat con el número guardado en la base.

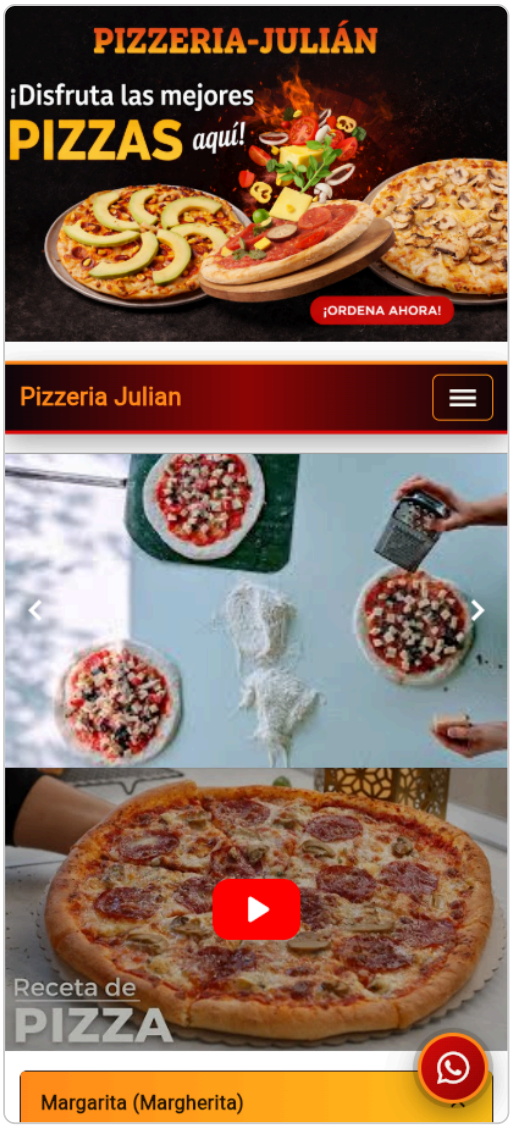


Figura 54. Pantalla Inicio.

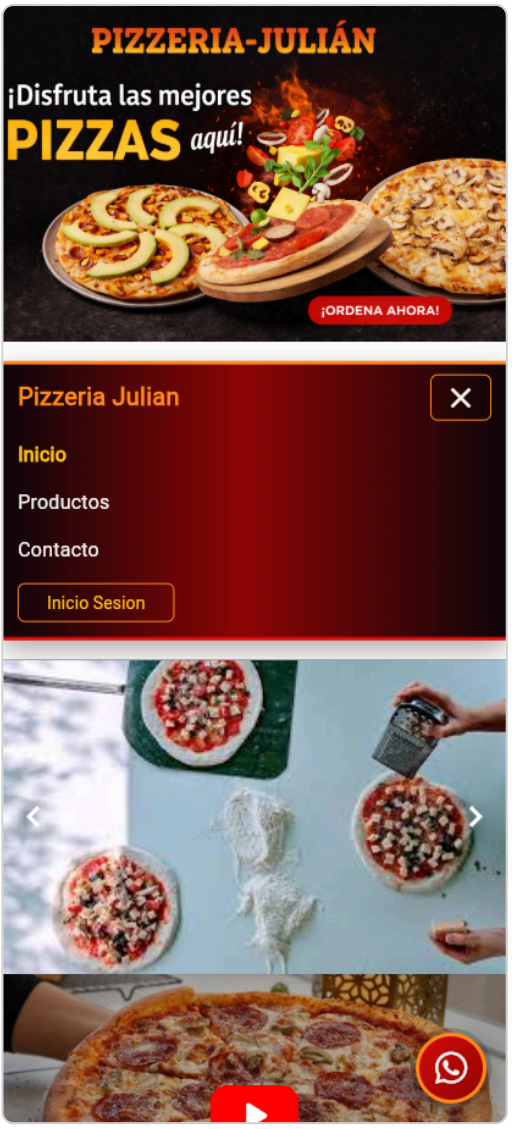


Figura 55. Menú desplegado.

35 Acordeón y secciones desplegables

El acordeón muestra cinco pizzas; al tocar una se abre y se cierra la anterior, como en la web. Los botones rojos despliegan la descripción de otras tres pizzas. Al final está el pie de página con las redes sociales y el menú.

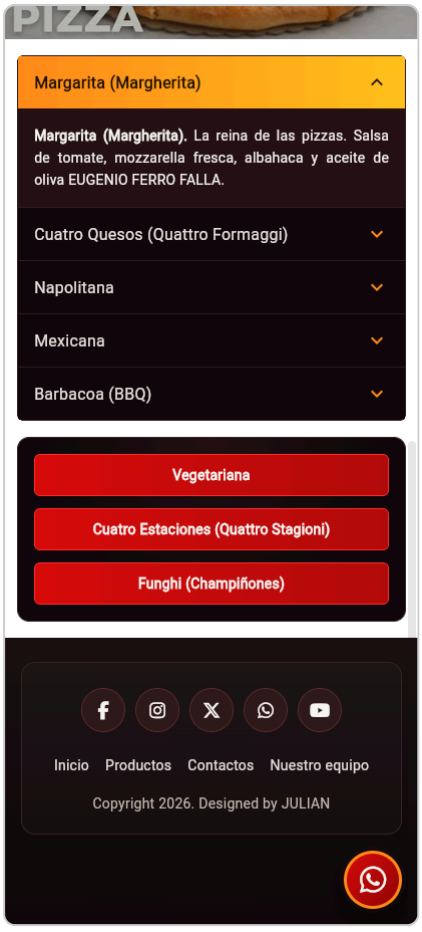


Figura 56. Acordeón (primer elemento abierto).

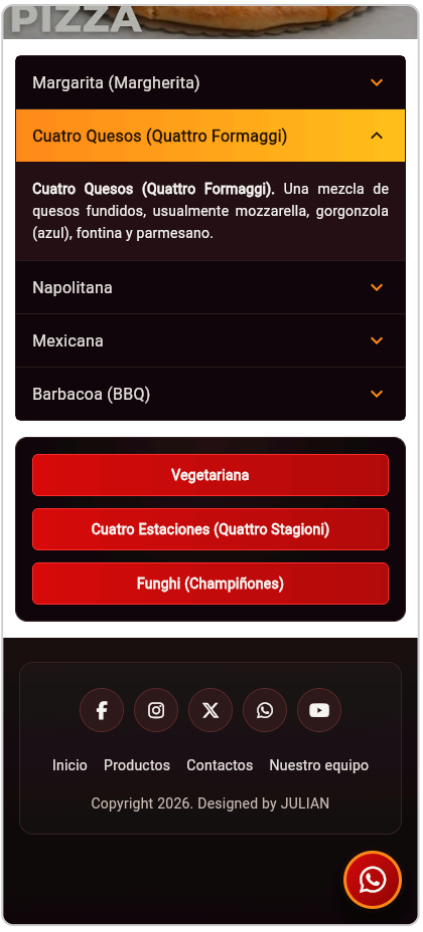


Figura 57. Otro elemento abierto.



Figura 58. Sección "Vegetariana" desplegada.

36 Pantalla Productos

Cada tarjeta tiene un carrusel de tres imágenes que cambia cada 3 segundos (también se puede mover con las flechas), el nombre, la descripción y el botón **Pedir ahora**, que abre WhatsApp con el nombre de la pizza.

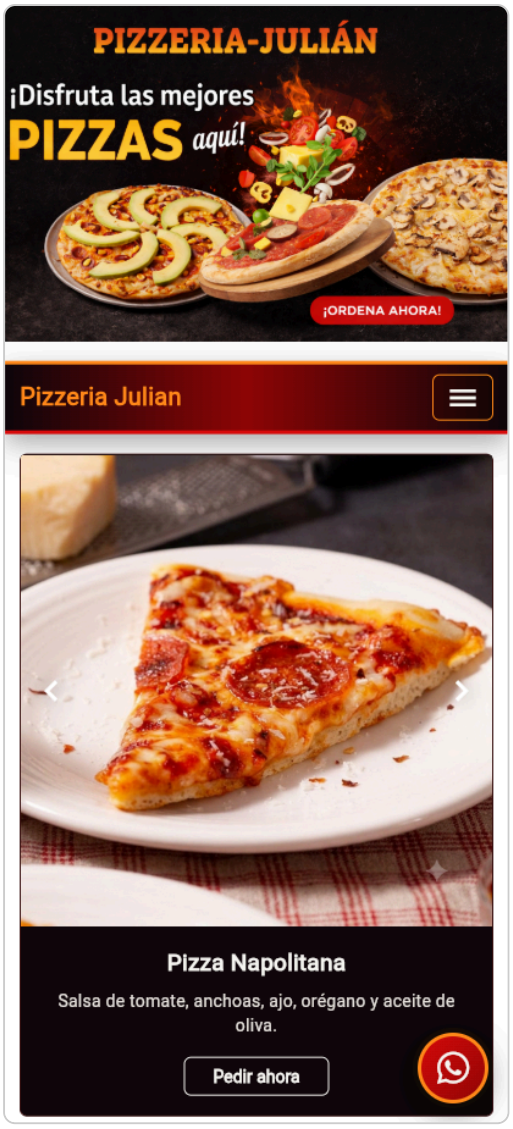


Figura 59. Pantalla Productos.

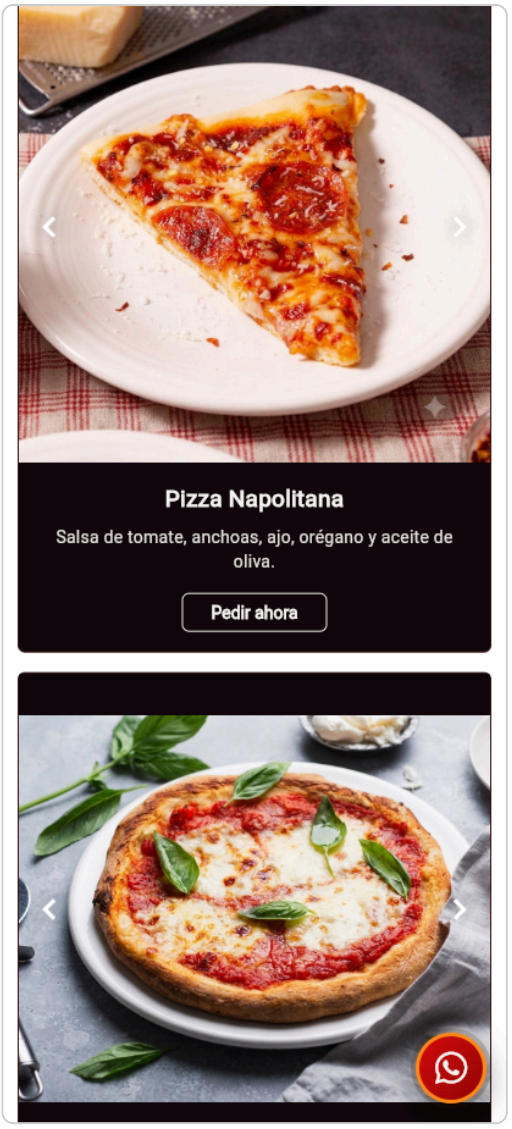


Figura 60. Tarjetas de productos.

37 Pantalla Contacto: mapa y formulario

Arriba se muestra el mapa de Google de la pizzería, y abajo el formulario. Si se intenta enviar vacío, la app marca en rojo los campos obligatorios. Al enviarlo, el mensaje se guarda en PostgreSQL y aparece el aviso verde.

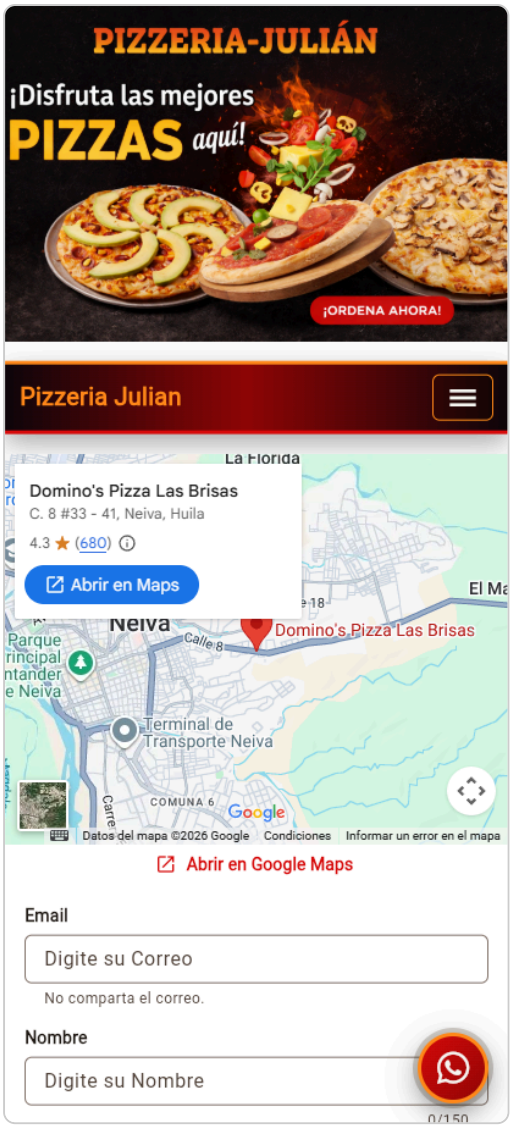


Figura 61. Mapa de Google en Contacto.

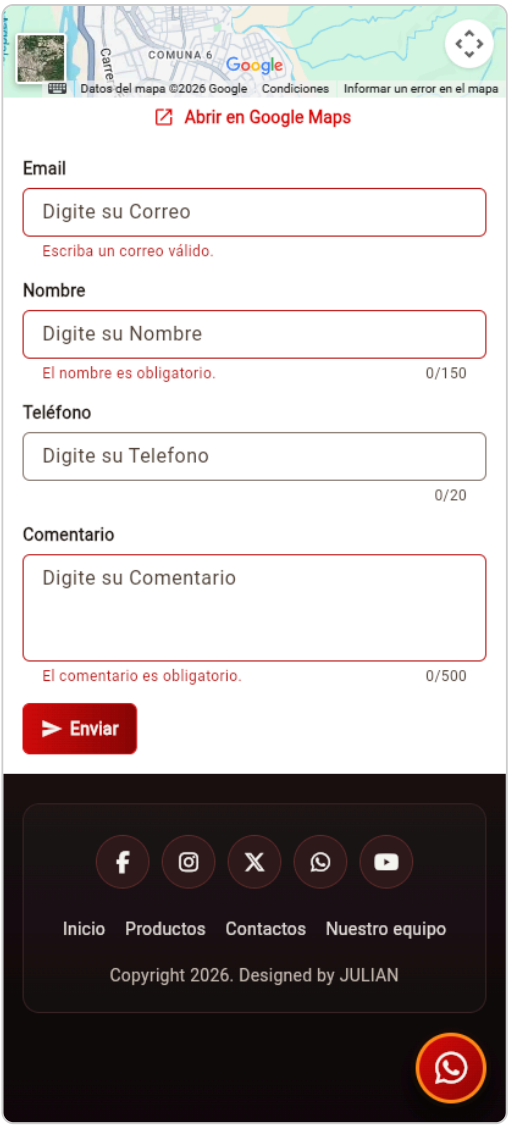


Figura 62. Validación de campos obligatorios.

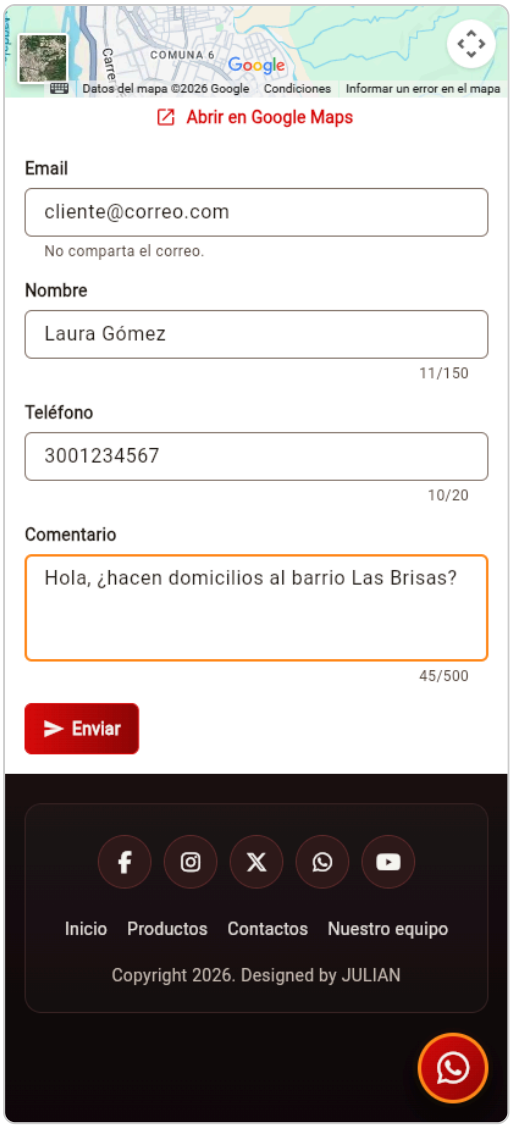


Figura 63. Formulario diligenciado.



Figura 64. Mensaje guardado correctamente.

El mensaje enviado desde la app queda en la tabla `formulario_contacto`:

```
▷ Windows PowerShell

PS D:\FLUTTER\flutter> docker exec pizza_julian_db psql -U postgres -d pizza_julian -c "SELECT id_formulario,
nombre_formulario, correo_formulario, mensaje_formulario FROM formulario_contacto ORDER BY id_formulario DESC LIMIT
3;"
 id_formulario |     nombre_formulario      |   correo_formulario    |                  mensaje_formulario
---------------+----------------------------+------------------------+---------------------------------------
---------
             6 | Laura Gómez                | cliente@correo.com     | Hola, ¿hacen domicilios al barrio Las Brisas?
             4 | CUALQUIERA                 | g46113026@gmail.com    | Hola checho estamos en el Tecnico Programando
con PHP.
             3 | ALBER DELFIN PEÑA ORTIGOZA | delfin.alber@gmail.com | Hola un saludo
(3 rows)
PS D:\FLUTTER\flutter>
```

Figura 65. El mensaje de "Laura Gómez" guardado en PostgreSQL.

> Cuando el archivo `.env` tiene la clave de aplicación de Gmail (`GMAIL_APP_PASSWORD`), el aviso dice "*Tu mensaje se envió correctamente*" y el mensaje también llega al correo de la pizzería. Para este manual se probó sin Gmail; por eso el aviso dice "*Tu mensaje se guardó correctamente*".

38 Iniciar sesión

En el menú, toque **Inicio Sesion**. Se abre la ventana con el contador de **30 segundos**: si se acaba el tiempo, la ventana se cierra sola. Si el usuario o la contraseña están mal, aparece un aviso rojo y la contraseña se borra. Tras 5 intentos fallidos, el inicio de sesión queda bloqueado 5 minutos.

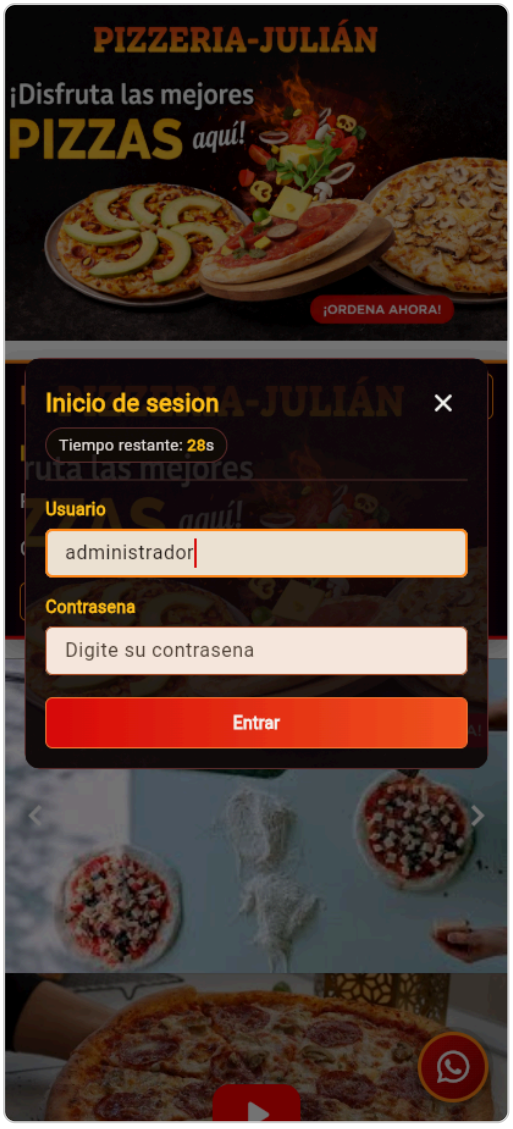


Figura 66. Ventana de inicio de sesión con contador.



Figura 67. Usuario o contraseña incorrectos.

39 Editor de contenido de Inicio

Al entrar con un usuario válido, se abre el editor de la sección donde estaba. En el editor de Inicio se ve el registro con su banner, el video y el WhatsApp, con los botones **Editar** y **Eliminar** (este último pide confirmación).

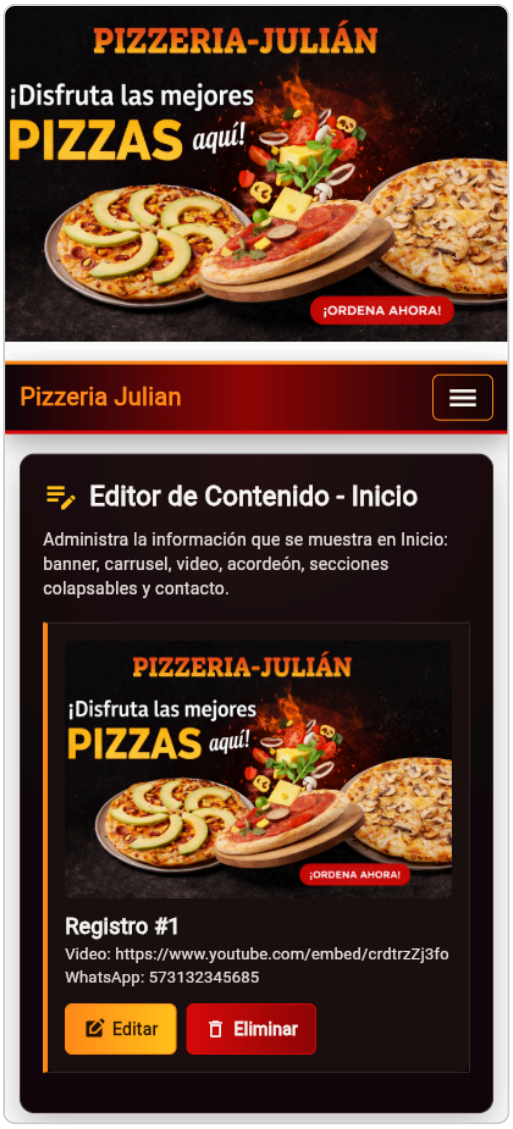


Figura 68. Editor de contenido de Inicio.

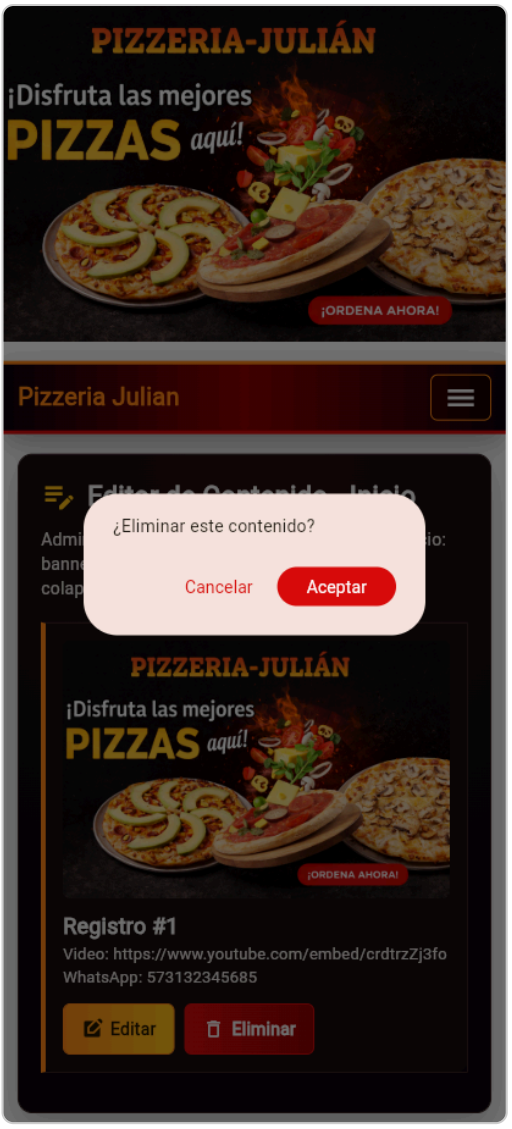


Figura 69. Confirmación antes de eliminar.

El botón **Editar** abre el formulario, organizado en cuatro grupos: **Banner y multimedia** (imágenes elegidas de la galería y enlace de YouTube), **Acordeón**, **Secciones colapsables** y **Contacto** (WhatsApp). Si se cambia el banner, al guardar se calcula de nuevo la paleta de colores de toda la app.

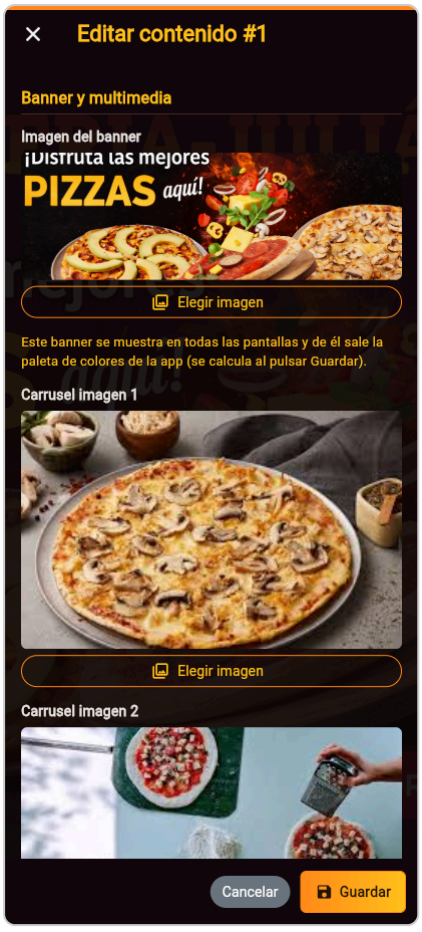


Figura 70. Banner y carrusel.



Figura 71. Textos del acordeón.

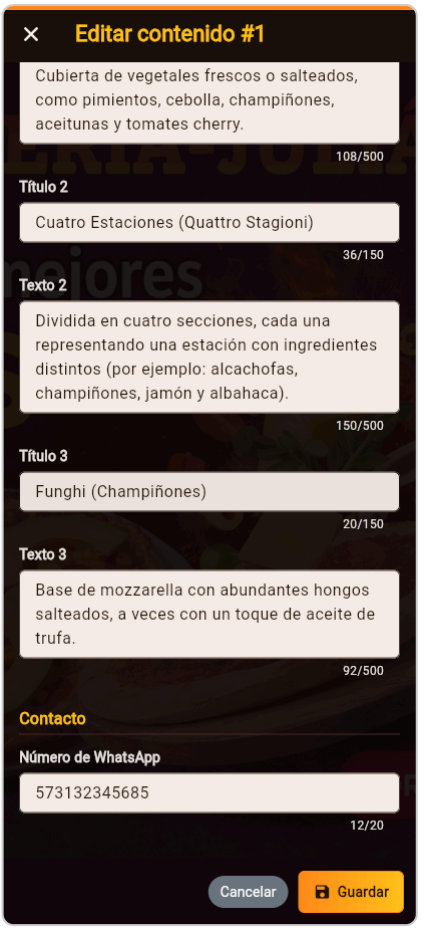


Figura 72. Secciones y WhatsApp.

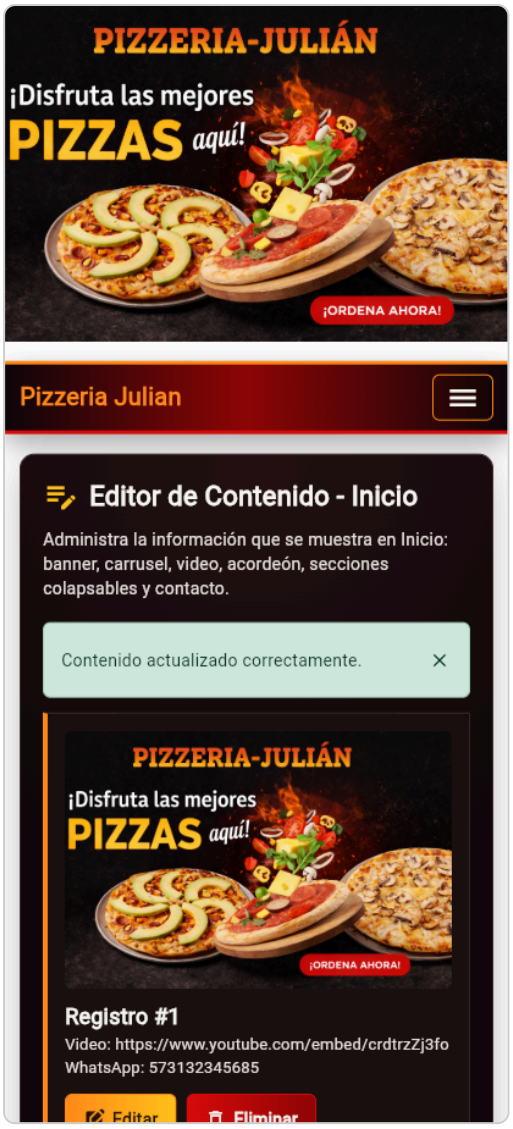


Figura 73. "Contenido actualizado correctamente."

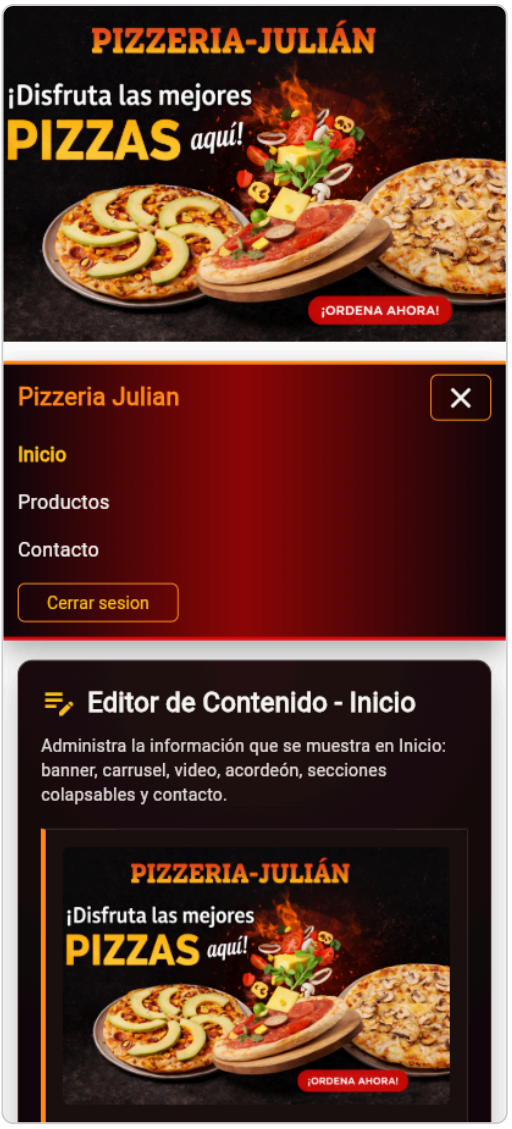


Figura 74. Menú del editor con "Cerrar sesion".

40 Editor de Productos

Muestra todas las tarjetas con **Editar** y **Eliminar**, y el botón **Agregar producto**. Cada producto necesita sus 3 imágenes (JPG, PNG, GIF o WEBP de máximo 5 MB), un título, un texto y el texto del botón.

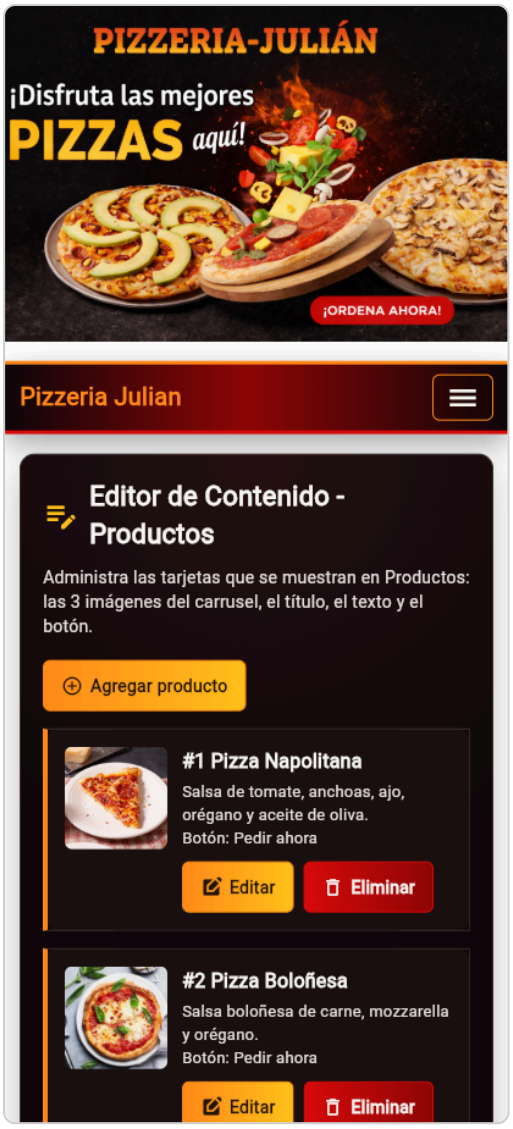


Figura 75. Editor de Productos.

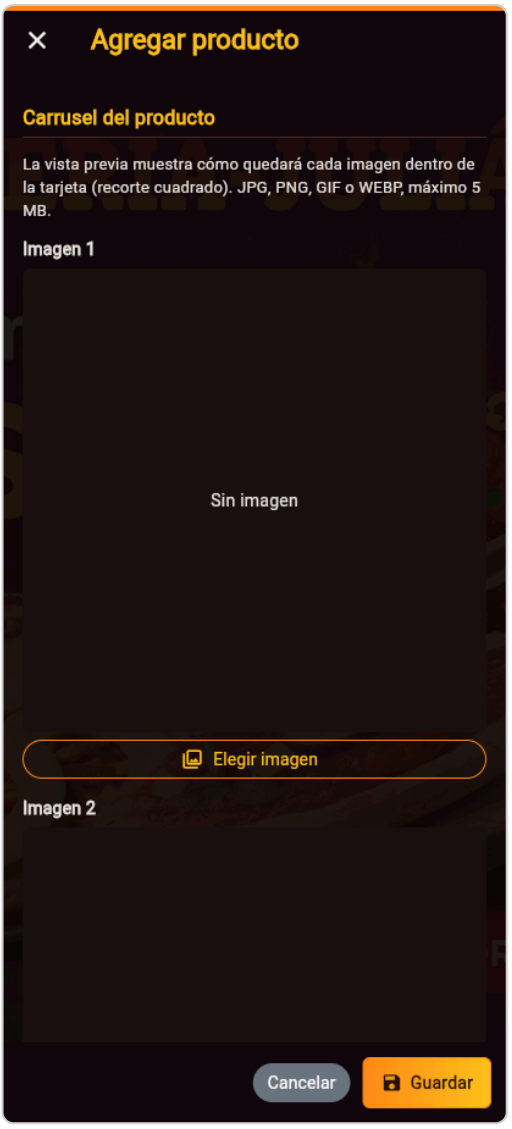


Figura 76. Formulario para agregar un producto.

Ejemplo: se editó el producto #1, se le agregó la palabra "Especial" al título y se guardó. El cambio quedó de inmediato en PostgreSQL y en la lista:

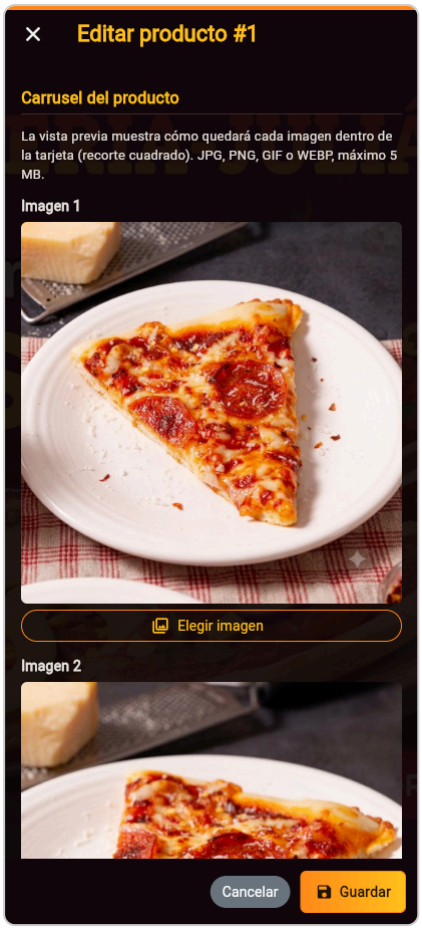


Figura 77. Editar producto #1.



Figura 78. Título modificado.

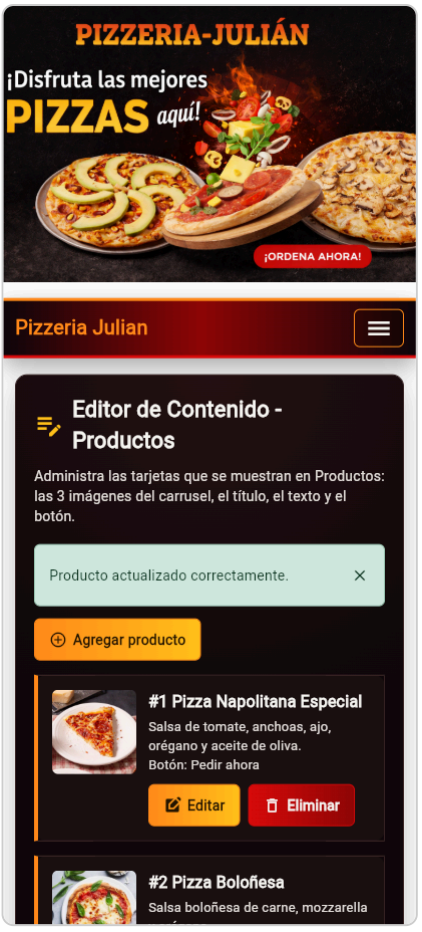


Figura 79. "Producto actualizado correctamente."

41 Editor de Contacto

Permite cambiar el mapa. En Google Maps busque el negocio y toque **Compartir → Insertar un mapa → Copiar HTML**; puede pegar el código completo o solo la URL, y la app toma la dirección correcta.

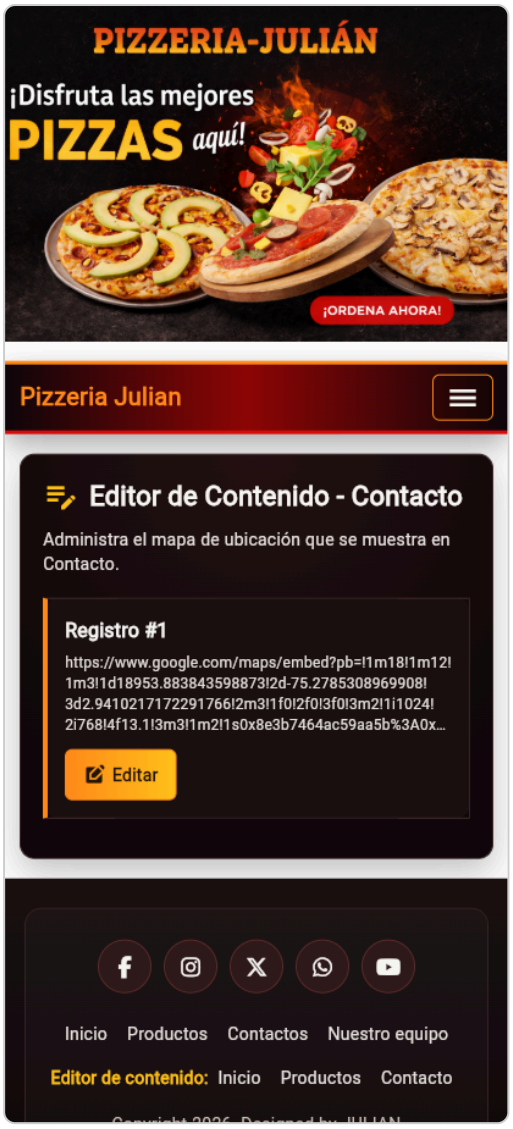


Figura 80. Editor de Contacto.

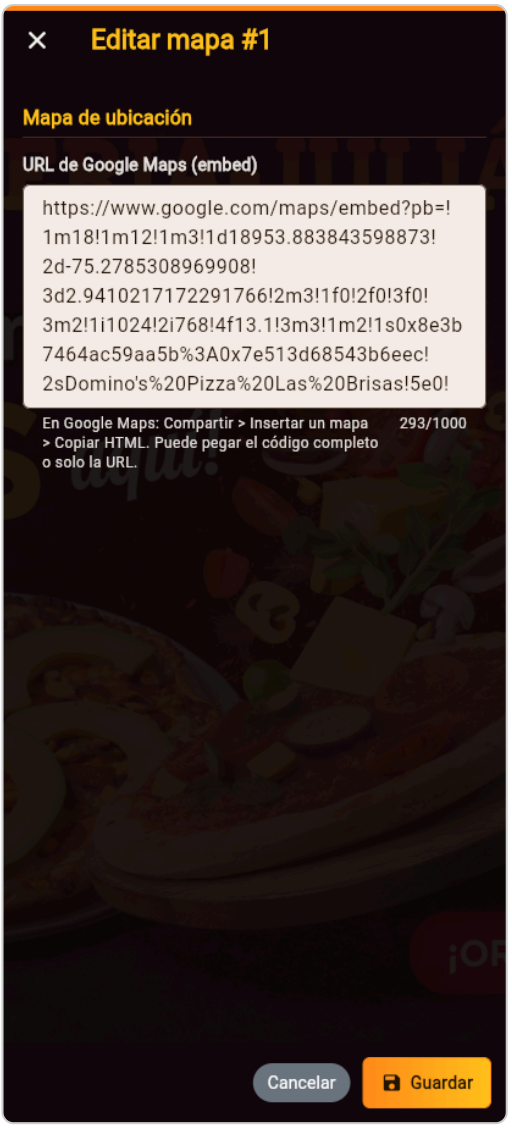


Figura 81. Formulario de la URL del mapa.

42 Cerrar sesión

En el menú del editor toque **Cerrar sesion**. La app borra el token en el servidor y vuelve a la pantalla Inicio pública. Por seguridad, el botón "Atrás" no regresa al editor.

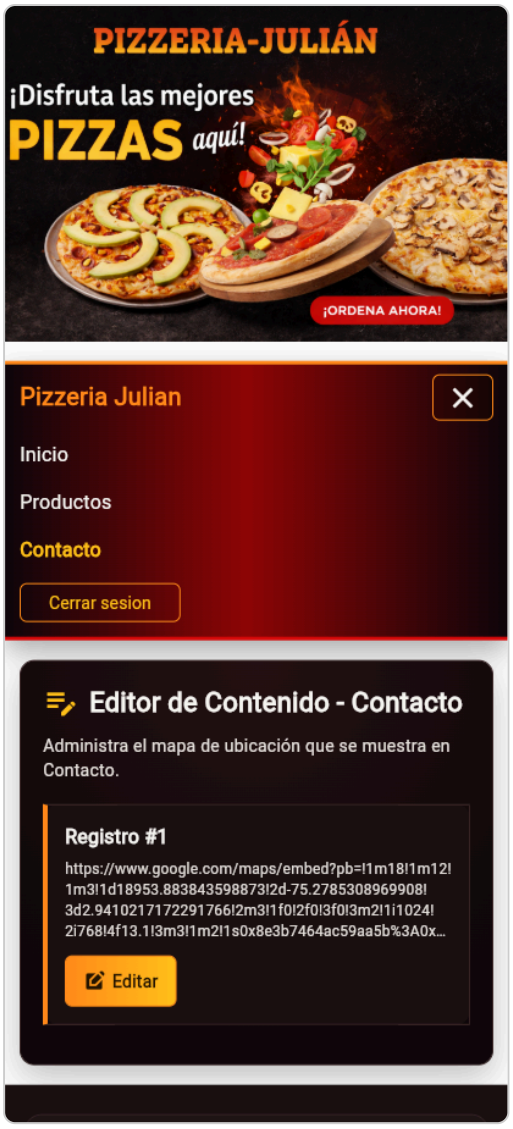


Figura 82. Botón "Cerrar sesion".

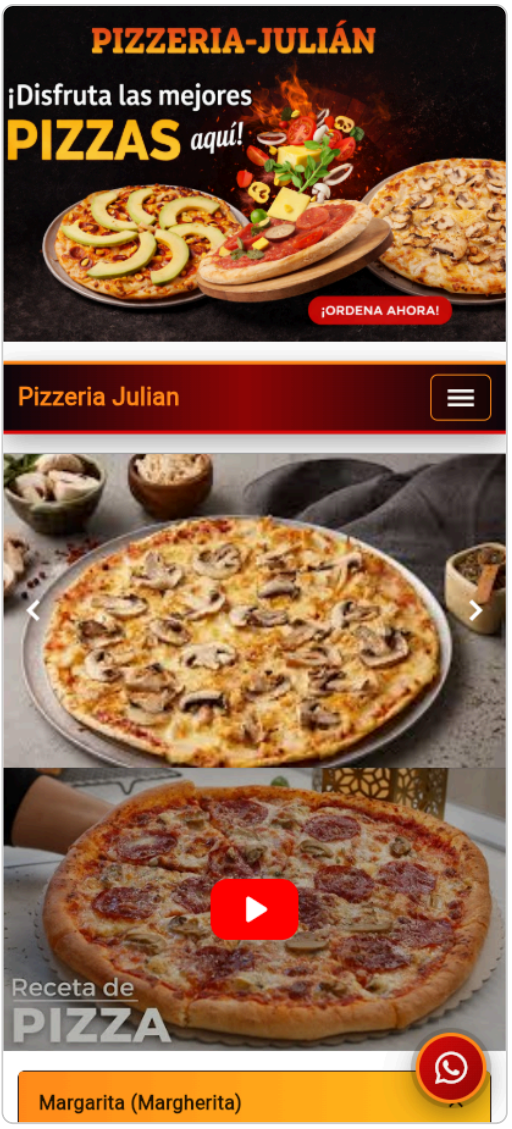


Figura 83. De vuelta en Inicio, sin sesión.

Parte 7. Celular, GitHub y uso diario

43 Compilar el instalador para Android (APK)

```
▷ Windows PowerShell
PS D:\FLUTTER\flutter\pizza_julian_app> flutter build apk --debug
Running Gradle task 'assembleDebug'...                            332.3s
✓ Built build/app/outputs/flutter-apk/app-debug.apk
PS D:\FLUTTER\flutter\pizza_julian_app>
```

Figura 84. Compilación del APK de Android.

Para un **celular real**, el servidor debe ser accesible por la red Wi-Fi:

1. Conecte el computador y el celular a la **misma red Wi-Fi**.
2. En PowerShell ejecute `ipconfig` y anote la **Dirección IPv4** (por ejemplo `192.168.1.10`).
3. Permita el puerto 8080 en el Firewall de Windows (cuando Windows pregunte, dé clic en **Permitir acceso** para Dart).
4. Compile indicando esa dirección:

```
flutter build apk --release --dart-define=API_URL=http://192.168.1.10:8080
```

El instalador queda en `build\app\outputs\flutter-apk\app-release.apk`. Páselo al celular y ábralo para instalarlo (Android pedirá permitir "instalar apps de origen desconocido"). En el emulador de Android Studio no hace falta indicar la IP: la app usa por defecto `http://10.0.2.2:8080`.

44 Guardar y subir el proyecto a GitHub

```
▷ Windows PowerShell
PS D:\FLUTTER> git add flutter
PS D:\FLUTTER> git commit -m "Agregar app movil Flutter de Pizza Julian con API Dart y base de datos PostgreSQL"
PS D:\FLUTTER> git push origin main
To https://github.com/delfinalber/FLUTTER.git
   77eeac5..f72c54e  main -> main
PS D:\FLUTTER> git log --oneline -5
f72c54e Agregar app movil Flutter de Pizza Julian con API Dart y base de datos PostgreSQL
77eeac5 Agregar proyecto pizza-julian
77284b3 first commit
PS D:\FLUTTER>
```

Figura 85. Commit y push al repositorio github.com/delfinalber/FLUTTER.

> El `.gitignore` evita subir el SDK de Flutter (unos 2 GB), los archivos `.env` con contraseñas y las carpetas `build`.

45 Uso diario: cómo abrir la app cada vez

1. Abra **Docker Desktop** y espere a que diga *Engine running*. El contenedor `pizza_julian_db` se enciende solo.
2. **Ventana 1 de PowerShell**, el servidor:

```
cd D:\FLUTTER\flutter\pizza_julian_api
$env:DB_PORT="5433"; $env:DB_PASSWORD="pizzajulian"
dart run bin/server.dart
```

3. **Ventana 2 de PowerShell**, la app:

```
cd D:\FLUTTER\flutter\pizza_julian_app
flutter run -d chrome --dart-define=API_URL=http://localhost:8080
```

4. Para terminar: **q** en la ventana 2 y **Ctrl + C** en la ventana 1.

> Si deja `DB_PORT=5433` y `DB_PASSWORD=pizzajulian` escritos en el archivo `.env`, no necesita la segunda línea de la ventana 1.

46 Solución de problemas frecuentes

Mensaje o síntoma	Causa	Solución
la autentificación password falló para el usuario «postgres»	La contraseña de `.env` no es la de PostgreSQL.	Use la contraseña de pgAdmin, o use Docker (puerto 5433, clave `pizzajulian`).
No fue posible conectar con PostgreSQL	Docker Desktop está cerrado o el contenedor está detenido.	Abra Docker Desktop y, en Containers, inicie `pizza_julian_db` (▶).
Solo se permite un uso de cada dirección de socket ... port = 8080	Ya hay un servidor abierto en el puerto 8080.	Cierre la otra ventana del servidor (Ctrl + C) o reinicie el computador.
La app dice "*Error de comunicación con el servidor*"	El servidor está apagado o la dirección `API_URL` está mal.	Encienda el servidor (paso 23) y revise la dirección.
'flutter' no se reconoce como un comando	El PATH no tiene la carpeta del SDK.	Repita el paso 3 y vuelva a abrir PowerShell.
Flutter requires Android SDK 36	Falta actualizar Android.	Android Studio → SDK Manager → Android SDK Platform 36 (paso 5).
El formulario de contacto no llega al correo	Falta la clave de aplicación de Gmail.	Escriba `GMAIL_APP_PASSWORD` en `.env` (clave de aplicación de Google, no la contraseña normal).

Fin del manual · Pizza Julián · 2026